

# What a Wonderful World: zkSNARKs in the Algebraic Group Model are Universally Composable

Gaspar Anthoine<sup>1,2</sup>, Dario Fiore<sup>1</sup>, and Mahak Pancholi<sup>1</sup>

<sup>1</sup> IMDEA Software Institute, Madrid, Spain

<sup>2</sup> Universidad Politécnica de Madrid, Madrid, Spain

**Abstract.** Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (zkSNARKs) are important cryptographic primitives critical in many real-world applications. zkSNARKs are not used in isolation but are deployed within a broader context in which other cryptographic protocols may be concurrently executed. Universal-Composability (UC) allows rigorous analysis of cryptographic primitives being used in such arbitrary contexts. A UC analysis is even more desirable for popular, well-audited, and heavily deployed zkSNARKs already being used in practice.

Prior works that study the UC security of existing zkSNARKs (without modifications) are either not modular, hence requiring case-by-case analysis for new proof systems, or have largely focused on zkSNARKs in the Random Oracle Model (ROM). The latter includes zkSNARKs with logarithmic proof sizes compiled from Interactive Oracle Proofs. This state of the art leaves out a large family of very efficient, often constant-size, zkSNARKs that rely on the Algebraic Group Model (AGM) and optionally on the ROM. This includes zkSNARKs compiled from Polynomial Interactive Oracle Proofs, such as Plonk and Marlin, among others.

In this work, we address the UC security for unmodified zkSNARKs that are proven secure in AGM (+ROM). Our approach is modular: we identify simple, and mostly standard properties on the underlying zkSNARK that imply UC security. We observe that checking these properties for existing zkSNARKs is a surprisingly simple task using the rigorous formulation of AGM from Jaeger and Mohan (CRYPTO’24). The simplicity and modularity of our framework makes it easy-to-use for concluding UC security of several zkSNARKs in the same setting. Concretely, using our framework we establish that Plonk and Marlin are UC secure without any overhead.

## 1 Introduction

Zero-knowledge Succinct Non-interactive Arguments of Knowledge (zkSNARKs) are one of the most useful cryptographic ideas that are being deployed in several real-world applications. Their prevalence necessitates optimal security guarantees. Furthermore, zkSNARKs are not used in isolation but are often composed with other cryptographic primitives within a larger application. In such settings, it is of utter importance to ensure that the interaction of zkSNARKs with concurrently-executed arbitrary protocols or their composition within the context of a bigger protocol do not compromise their security.

The Universal Composability (UC) framework [21] addresses exactly this problem. It enables the modular design and analysis of cryptographic protocols and guarantees security even against adversaries that run multiple concurrent sessions and mount arbitrary attacks.

Prior attempts at achieving UC-secure zkSNARKs usually lift or transform an existing zkSNARK to make it secure in the UC framework [5, 37]. These approaches substantially alter the underlying scheme, incurring significant overhead, and do not address many of the existing (unaltered) zkSNARKs commonly used in practice.

This paper focuses on UC security of existing zkSNARKs without modifications. Necessary ingredients for UC security of zkSNARKs are *straight-line* (SLE) and *black-box* extraction (BBE), and *simulation-extractability* (SIMEXT). Informally, a proof system is SLE if a valid witness can be extracted without interacting with the adversary (hence no rewinding); it is BBE if the extractor works without knowing the code of the adversary; and it is SIMEXT if it is *non-malleable*, i.e., an

adversary cannot maul an existing proof to forge a new proof on some statement without knowing the corresponding witness.

Groth [42] showed in a modular manner that if a (non-succinct) NIZK in the CRS model satisfies the aforementioned properties then it is UC-secure in the CRS-hybrid model. Succinct proof systems with straight-line black-box extraction, however, cannot exist in the plain CRS-model alone [20, 39]; instead they are extractable in some idealised model. Such models include the random oracle model (ROM), or an idealised group model such as the generic group model (GGM<sup>3</sup>) [59, 61] or the algebraic group model (AGM) [34, 45]. Nonetheless, in such idealised models, several modern zkSNARKs have recently been shown to satisfy SLE, SIMEXT, and sometimes BBE [32, 33, 38, 49]. This brings up the natural question:

*Following [42], can we prove in a modular manner the UC security of existing zkSNARKs proven extractable in an idealised model?*

To further motivate this question, let us briefly summarize the state of the art on SLE of existing zkSNARKs. We can broadly classify most of them in two categories.

**Category 1: zkSNARKs with SLE in the ROM.** This includes purely ROM-based zkSNARKs, such as Killian-Micali [48, 57], and the ones compiled from Interactive Oracle Proof (IOP) using a Merkle tree [10–12]. These zkSNARKs achieve logarithmic proof sizes at best.

The zkSNARKs in this category happen to naturally satisfy BBE and SLE via observability in the ROM. Leveraging this observation, two recent works [22, 26] addressed the UC security of these zkSNARKs, using an hybrid UC model where the ROM is formulated as a global programmable functionality (global-ROM). Concretely, both works show UC security for the Kilian–Micali zkSNARK [56], while [26] additionally considers zkSNARKs obtained by compiling an IOP with Merkle-tree commitments via the BCS compiler [12].

**Category 2: zkSNARKs with SLE in an idealised group model (+ROM).** This includes constructions such as [29, 43, 51], zkSNARKs obtained from polynomial interactive oracle proofs (PIOPs) using extractable polynomial commitment schemes [17, 19, 27, 36, 54, 54], as well as transparent zkSNARKs relying on discrete-log-type assumptions [14, 15, 60]<sup>4</sup>. These schemes are also among the most efficient and succinct systems we have today, with many zkSNARKs in this category having constant proof size unlike those in Category 1.

For these zkSNARKs, the essential properties for UC crucially depend on an idealised group model rather than the ROM. Hence, techniques in [22, 26] are not applicable here.

The recent work of Bobolz et al. [13] is the first to address the UC security of a proof system in Category 2. They do so in a hybrid UC model where the GGM is formulated as a global group functionality (global-GGM). Now, via observing group queries, one can recover BBE and SLE as in Category 1. Concretely, they prove that Groth16 [43] is UC-secure without any modifications, under this new model.

While [13] provides a clear blueprint for establishing UC security in the GGM, it has three major limitations. First, it is a purely GGM-based model while several zkSNARKs additionally require the ROM. Second, it requires the prerequisite security properties of the zkSNARK to be defined and proven in GGM, whereas the common practice is to establish these in the AGM (+ROM). This means that to show UC security for Groth16, the analysis of SIMEXT has to be rewritten in the GGM, and cannot simply invoke the existing theorem proven in the AGM [7]. And finally, their analysis is not modular. In particular, the framework does not offer a modular way to leverage

<sup>3</sup> Whenever we refer to GGM, we mean Shoup’s GGM introduced in [61].

<sup>4</sup> We include these zkSNARKs in Category 2 since SLE very likely requires the algebraic group model (for example, see [40] for an SLE analysis of Bulletproofs [15])

existing standalone security analyses, and explicitly leaves open the problem of a general *UC lifting theorem* that would allow one to derive UC security from simpler, game-based notions.

This lack of modularity is particularly unsatisfactory given that many existing zkSNARKs in Category 2 already satisfy, in the AGM, the key properties that seem necessary for UC security [32,33,38,49]. From this perspective, these zkSNARKs look already “close” to being UC-secure, and one would ideally like to leverage their well-established AGM-based security analyses directly.

Hence the goal of this work is to study the following question and propose a modular framework to establish it:

*If a zkSNARK satisfies SLE and SIMEXT in the AGM(+ROM), then is it UC secure in the presence of global-GGM+(global-ROM) functionality?*

## 1.1 Our Results

Our main result is answering the aforementioned question positively, i.e., zkSNARKs satisfying straight-line simulation-extractability in the AGM+ROM are UC-secure.

1. **UC lifting theorem:** The work of [13] introduces a very useful UC framework meant to work well with AGM-based zkSNARKs. However, it is quite non-trivial to use since it essentially requires most proofs to be redone from scratch within their framework. Our main result leverages the observation that AGM-based zkSNARKs are *already* very close to UC: We identify simple properties for zkSNARKs in the stand-alone model formulated in GGM (+ROM) that imply UC security in the presence of a global-GGM +(global-ROM) functionality. Surprisingly, these properties closely mirror standard notions for zkSNARKs. This has the added benefit that potentially one does not need to prove *new* properties to obtain UC security.

**Theorem 1.** *(Informal.) If a zkSNARK is complete, non-interactive zero-knowledge, and straight-line and black-box simulation-extractable in GGM (+ROM), then it is UC-secure in the presence of global-GGM (+non programmable global-ROM) functionality.*

Given that most analyses exist in AGM (+ROM), the next question is how to lift these analyses to GGM without having to redo them. We do this in two steps:

2. **Lifting guarantees in AGM+ROM to “type-safe” (GGM+ROM):** As a stepping stone toward a general lifting theorem, we first show how to lift such guarantees to an intermediate model, called the *type-safe*<sup>5</sup> (GGM+ROM). While [45] presents a similar lifting result, their framework does not allow algorithms to access a random oracle and is therefore not directly applicable to the zkSNARK constructions considered in this work. We observe that a similar hierarchy to that of [45] can be defined even in the case where algorithms require access to ROM. At first sight, it is natural to ask:

*How can a random oracle taking a bitstring as input be used in a restricted model where group elements do not even have a concrete representation?*

To circumvent this limitation, we first introduce a new type-safe random oracle and show that it can be canonically translated into a “classical” random oracle. Leveraging these observations, we show how existing security analyses carried out in the AGM+ROM can be largely reused to establish security in (our variant of) type-safe (GGM+ROM).

---

<sup>5</sup> A type-safe GGM is closer to Maurer’s GGM [55] where group elements are stored in a table and group operations can be performed only via their position in the table. As such no concrete group representation is available.

**Theorem 2.** *(Informal.) The security of a primitive proven in the AGM+ROM can be lifted to the type-safe (GGM+ROM) provided that the reduction is (type-safe) generic.*

3. **Type-safe (GGM+ROM) and GGM+ROM equivalence:** The work of [62] establishes an equivalence between the type-safe GGM and GGM for single-stage adversaries, and for primitives that can be expressed in the type-safe model. Building on similar observations, we show an analogous equivalence between our variant of the type-safe (GGM+ROM) and GGM+ROM. Combining this equivalence with Theorem 2 yields a clear framework for establishing stand-alone security properties in GGM+ROM. Finally, applying our UC lifting theorem Theorem 1, one can conclude UC security.

**Theorem 3.** *(Informal.) For primitives that exist both in type-safe (GGM+ROM) and GGM+ROM, security against single-stage adversaries is equivalent.*

**How to use our framework.** We apply our framework to show that Plonk [36] and Marlin [27] are UC-secure in the presence of global-GGM and (non-programmable) global-ROM functionalities. The work of Faonio et al. [33] establishes that both Plonk and Marlin, without modifications, are SLE SIMEXT in the (non-programmable) ROM and the AGM. Starting from this result, we first apply our lifting result (Theorem 2) to SIMEXT of the same protocols in our variant of the type-safe (GGM+ROM). Using the equivalence result (Theorem 3), we then conclude that these zkSNARKs are SLE SIMEXT in GGM+ROM. Completeness and NIZK follow similarly. Since these properties are the prerequisite for our UC lifting theorem (Theorem 1), we ultimately obtain UC security for Plonk and Marlin in the presence of global-GGM and (non-programmable) global-ROM functionalities.

## 2 Technical Overview

### 2.1 UC lifting theorem

Our goal is to identify properties in the stand-alone setting that are sufficient for proving UC. In this sense, we are closer to [26] in our methodology than [13, 22]. While ideally, one would like to model AGM behaviour in UC to be closer to the stand-alone setting, the UC-AGM model [1] is not suitable for non-interactive extractable primitives like zkSNARKs (see Appendix A). The next best option is to model GGM behaviour in UC as done in [13].

**Proving UC security via game hops.** We give a short recap on the common approach for showing UC security for zkSNARKs in the literature. UC security is proven by showing indistinguishability between a real-world and an ideal-world experiment. For a zkSNARK  $\Pi$ , the real-world experiment is defined as: the honest parties in a session  $sid$  execute the (prove, verify) protocols specified in  $\Pi$  while having access to the global and (if any) local functionalities. While in the ideal-world experiment, no protocol is executed. Instead the honest parties access an ideal NIZK functionality that, at a very high level, behaves as follows: On receiving a prove command from an honest party, run a simulator algorithm on just the statement to generate a proof string (which ensures zero-knowledge). On receiving a verify command from an honest party, run the extractor on the proof string, and check if the output is a valid witness. If it is valid, then output 1, otherwise output 0 (which ensures knowledge-soundness). In addition to this, a simulator algorithm is responsible for simulating the view of the corrupted party in  $sid$ . This includes simulating interaction with honest parties (as in the real world) and access to the local functionality.

A natural approach towards proving UC-security of a zkSNARK is via doing standard game-hops (some examples where this shows up are [26, 37, 42]) that go as follows:

- $\text{Hyb}_0$ : This is the real-world experiment in UC.
- $\text{Hyb}_1$ : The prove interface is replaced by the zero-knowledge simulator.
- $\text{Hyb}_2$ : The verifier interface is swapped by running the extractor on the proof string to obtain a witness and checking whether the witness is valid or not. This is exactly like the ideal world UC experiment.

To argue indistinguishability between each hop, one would typically invoke an underlying stand-alone property (i.e. zero-knowledge for  $\text{Hyb}_0$ - $\text{Hyb}_1$ , and simulation-extractability for  $\text{Hyb}_1$ - $\text{Hyb}_2$ ).

Consider the reduction for  $\text{Hyb}_0$ - $\text{Hyb}_1$  hop. For simplicity, assume that the parties only have access to a global-ROM (and not global-GGM). Assume that there exists an adversary  $\mathcal{Z}$  who can successfully distinguish between  $\text{Hyb}_0, \text{Hyb}_1$ . Then, we build an adversary  $\mathcal{A}$  given black-box access to  $\mathcal{Z}$  against the zero-knowledge property. At a high level,  $\mathcal{A}$  runs  $\mathcal{Z}$  internally and is responsible for simulating its view. This includes answering prove, verify commands, and simulating accesses global-ROM, while interacting with the challenger in the zero-knowledge game in the presence of a standard ROM.  $\mathcal{A}$  can simply forward the prove and verify commands to the underlying game and return the responses. To simulate responses to global-ROM given access to standard ROM,  $\mathcal{A}$  proceeds as follows: forward all  $\text{sid}$  queries to the standard ROM in the underlying game, simulate all  $\text{sid}' \neq \text{sid}$  queries locally, keep a local record of observable queries (this is possible since it sees all queries made by  $\mathcal{Z}$ ), and responds appropriately when asked for the observable set<sup>6</sup>. Since global-ROM has a very easy domain separation and  $\text{sid}'$  responses are independent of  $\text{sid}$ ,  $\mathcal{A}$  perfectly simulates  $\mathcal{Z}$ 's view. Hence, any distinguishing advantage of  $\mathcal{Z}$  implies the same advantage for  $\mathcal{A}$  in distinguishing real proofs from simulated ones.

**Extending to group-based zkSNARKs.** For group-based zkSNARKs, we not only need to model a global-ROM but also a global-GGM functionality. Global-GGM models Shoup's GGM [61] as a global functionality. In Shoup's GGM, the group is modelled as a random mapping of  $\mathbb{Z}_p$  to bit strings, and group operations can only be performed via queries to the GGM oracle.

The game hops follow similarly as above. However, building a reduction to an underlying property requires care. Since group elements are not independent for each session the domain separation is not as easy as in the case of global-ROM. As a result, in our reduction, we can no longer simulate  $\text{sid}'$  group queries locally, but all queries regardless of the session must be forwarded to the GGM oracle in the underlying game. Here, an issue arises when trying to simulate the observability interface of the global-GGM functionality. To understand the issue, first consider the mechanism of observability defined in [13].

In [13], every generator for some session  $\text{sid}$  is associated with a formal variable  $X_{\text{sid}}$ . A group element that is the result of a group operation now has an associated polynomial: for example, a group operation of the form  $h \leftarrow g_1 + g_2$  has the polynomial  $X_1 + X_2$  associated with it, where  $X_1, X_2$  are the formal variables previously defined for  $g_1, g_2$ , respectively. A group operation is labelled as illegal and hence observable if the polynomial associated with the result happens to have any foreign session variables  $X_{\text{sid}'}$  or a constant term. In short, operations involving other session's generators become observable.

In our reduction, the group operations performed by the algorithms in the underlying game (i.e., prove and verify interfaces) will not be visible to  $\mathcal{A}$ . This means that, while simulating observability for global-GGM,  $\mathcal{A}$  might not be able to classify legal vs illegal queries, or might classify them differently than if it had the full representation. An example of this is: Let  $g_{\text{sid}'}$  be the generator of  $\text{sid}'$ . On receiving a prove command from  $\mathcal{Z}$ ,  $\mathcal{A}$  forwards it to the prove interface of the underlying

<sup>6</sup> Global functionalities often have an OBSERVE interface that allows to observe “illegal” queries that  $\mathcal{Z}$  might make to force the simulator in  $\text{sid}$  to fail.



game, and receives a proof string  $(\pi_1, \dots, \pi_n)$  in return, where assume that  $\pi_1, \pi_2$  are group elements. Later,  $\mathcal{Z}$  makes a group query of the form  $g_{sid'} + a \cdot \pi_1 + b \cdot \pi_2$ . Looking at it naively,  $\mathcal{A}$  classifies this group operation as illegal since it contains group elements from different sessions (and because it does not know the polynomials associated with  $\pi_1, \pi_2$ ). However, with the full polynomial representation of  $\pi_1, \pi_2$  perhaps the *sid* terms cancel out, and hence will not be classified as illegal in the hybrid game.

We observe that the (symbolic) group oracle provides a natural mechanism to access the underlying polynomial representation. This means that maintaining partial observability is not a problem for  $\mathcal{A}$ . It can still determine observability of the future group elements by accessing the underlying polynomial representation via appropriate group queries to the GGM oracle. This allows it to verify if a query cancels out the “foreign” variables associated with other sessions *sid'*. Concretely, it makes the group queries corresponding to the operation  $a \cdot \pi_1 + b \cdot \pi_2$  and checks if the result equals to the canonical generator (hence the *sid* terms cancel out). If yes, then classify as legal, and otherwise mark as observable.

A related issue arises when reducing to simulation-extractability in the last hop. In the case of global-ROM alone, the argument is that the extractor in the underlying game has an identical view as that of the extractor in the hybrid (only queries related to *sid*). Hence, they succeed or fail with the same probability. However, in our reduction, the extractor in the game has full view of the queries made to the group oracle (since  $\mathcal{A}$  forwards everything), while in the hybrid game it only has *sid* related views. We argue that if the generators for each session are chosen independently and if whenever the honest protocol samples a group element it samples it with high min-entropy, then the additional information available in the underlying game is not more useful. Hence, the success probability can now be related identically.

What we have shown above is that the usual game-hop strategy follows through even in the case of global-GGM with mild assumptions on the zkSNARK, and without drastically modifying the game-based properties. The game-based properties must exist in GGM+ROM for our reduction to work. However, most known simulation-extractability analyses exist in the AGM+ROM. To avoid having to redo analysis in Shoup’s GGM (as is done in [13] for Groth16; simulation-extractability for Groth16 was already known in the AGM in a previous work [7]) and avoid subtle security gaps [64], we present a framework for lifting proofs in the AGM+ROM to Shoup’s GGM+ROM next.

## 2.2 AGM+ROM to GGM+ROM lifting

**Type-Safe model of computation.** Maurer [55] introduced an alternate GGM framework where the group elements are stored in a table and group operations are abstracted as having only access to *handles* representing an index in the table kept by an oracle. Subsequently in a seminal work, Zhandry [62] showed that relying on such a table might lead to undesirable behaviours and introduced the notion of Type-Safety using a circuit representation where handles are wires containing values in  $\mathbb{Z}_p$ .

The notion of type-safety was further formalized in [45] using a typed pseudocode representation of the algorithms. In this framework, each variable in the algorithm is assigned a data type;  $\langle \text{bit} \rangle$  for a bit string, and  $\langle \text{elem} \rangle$  for a group element. Data types restrict how an algorithm can interact with specific variables. Additionally, they have access to functions that perform group operations by accepting  $\langle \text{elem} \rangle$  variables as input, a function  $\sigma : \langle \text{elem} \rangle \rightarrow \langle \text{bits} \rangle$  that outputs a concrete representation of a group element, and  $\sigma^{-1} : \langle \text{bit} \rangle \rightarrow \langle \text{elem} \rangle$  that outputs a group element given its representation. In “type-safe GGM”, algorithms are only allowed to access group operation functions (meaning that they cannot obtain concrete representation for group elements); while in “type-safe

AGM” they additionally have access to  $\sigma$  but not to  $\sigma^{-1}$  (meaning that they cannot generate group elements locally without using functions for group operations).

They use their framework to rigorously formalize common intuitions about the AGM (we will call this the type-safe AGM). Under this formalization, they characterize conditions under which security in type-safe AGM can be lifted to type-safe GGM. The combination of [45] and [62] already lays the necessary groundwork for links between variants of AGM and GGM. There are a few things missing to be applicable to zkSNARKs, which we discuss next.

**Type-Safe Model with Random Oracle.** Many cryptographic primitives that can be expressed in Shoup’s GGM cannot even be expressed in the type-safe GGM [41, 62] because in the latter model the algorithms do not know group element representation. One type of computation that cannot be expressed is random oracle queries. To make a query to a standard ROM, the zkSNARK algorithm must use the concrete group element representations [41]. This also holds for the type-safe pseudocode framework of Jaeger and Mohan [45].

We resolve this by introducing a type-safe variant of random oracle. Providing a type-safe algorithm with access to a full random oracle that accepts group elements as input may raise concerns. Specifically, *what prevents the algorithm from using the random oracle to derive an encoding for a group element?*

While this is indeed the case if we see the oracle as a random injection  $\hat{\sigma}$ , we prove in Section 6.1 that it is possible to define a similar hierarchy as the one of Jaeger and Mohan [45] for algorithms with access to type-safe random oracle. Overall the argument relies on the fact that, as long as the algorithm can’t compute the inverse  $\hat{\sigma}^{-1}$ , group elements will only come from group queries similarly to the Type-Safe AGM and thus preserves the model’s security guarantees.

**Type-Safe Model to Shoup’s GGM.** Zhandry [62] showed that the two models are not equivalent as was formerly believed. Security in Shoup’s GGM is strictly stronger for games where the adversary is split into several stages without sharing full internal state. On the other hand, for “single-stage” adversaries, i.e., adversaries that pass full internal state between stages, and for primitives that can be expressed in the type-safe GGM, the security in both models of GGM is equivalent.

We complete the sequence of implications by showing a similar equivalence between the new type-safe model with access to type-safe random oracle introduced in this work, and Shoup’s GGM with access to a standard random oracle. As a result, the type-safe random oracle turns out to be a stepping stone towards proving security in the standard random oracle model, and is never used by the final algorithms.

## 2.3 Plonk and Marlin are UC-secure

As an application of our framework we show that Plonk and Marlin are UC-secure. The main “non-blackbox” task here is to check that the reductions (in [33]) to a computational problem are all type-safe generic. Intuitively since such reductions are written in the algebraic group model, they very likely satisfy the formal requirement as well. Nonetheless, in Appendix H, we reason that this is indeed the case. After this, we can in a black-box manner lift these results all the way up to UC.

## 2.4 Limitations

An important criticism of the generic group model is that it allows to prove certain (contrived) schemes secure that cannot be securely instantiated under any group in the standard model [30]. Moreover, security in GGM only holds against generic adversaries. What would be ideal is security against standard adversaries since a known class of sub-exponential time attacks on the discrete-log assumption are via index calculus algorithms which make use of the specific group structure. However,

composable guarantees in the GGM at the very least confirm that any attack on the system must inherently rely on the group structure.

## 2.5 Related Works

We present a more complete survey of related-works in [Appendix A](#).

**UC-secure zkSNARKs in global-ROM.** Ganesh et al. [37] were the first to present a UC-secure zkSNARK with constant-sized proofs in the CRS-model. Badertscher et al. [5] build upon [37] to achieve a UC zkSNARK in the transparent model. Both works are in the non-programmable global-ROM, and present a compiler that can lift an underlying zkSNARK with appropriate properties to a UC-secure one. The works of [22, 26] study UC security of existing zkSNARKs without modifications. In the programmable global-ROM (programmability seems necessary for such unmodified zkSNARKs), both show that Killian-Micali zkSNARK is UC-secure, while [26] additionally shows UC-security for certain IOP-based zkSNARKs obtained via the BCS transform [12]. Their constructions are secure against adaptive adversaries.

The work of [26] is closer to our work in terms of methodology as they too aim to identify stand-alone properties that imply UC security. However, they are only able to realise a weaker NIZK functionality. Informally the weaker functionality is not required to extract if the statement has already been proven in this session. On the other hand, [22] models all subprotocols directly in UC, for example, by introducing functionalities for non-interactive (vector) commitment schemes, and building a UC-secure zkSNARK from these.

For simplicity, we only consider static adversaries in this work, leaving the formalisation as an open problem.

**UC-secure zkSNARKs in global-GGM.** To tackle AGM-based zkSNARKs, Bobolz et al. [13] present a new UC framework modelling the group as a global generic functionality with restricted observability. Under this new model, they prove Groth16 [43] to be UC-secure. Since Groth16 is malleable, it can only realise a weak form of NIZK functionality, while in this paper we aim for the standard NIZK functionality.

**AGM to GGM lifting results.** A “lift” of a security reduction allows one to infer hardness in a target model based on a reduction against adversaries in a source model. First introduced in the original AGM [34] to infer security in the GGM from an AGM reduction, a subsequent counter-example [64] demonstrated that this implication is not guaranteed. This motivated the work of [45] that established a formal framework to derive a hierarchy of idealised models.

## 3 Preliminaries: Stand-Alone Properties

**General notational conventions.** We denote all stand-alone algorithms with font-type  $\mathcal{A}$ , stand-alone oracles with  $\mathcal{O}$ . The UC algorithms are boldfaced, while the functionalities calligraphic. For a function  $\tau : A \rightarrow B$ , we write  $\text{img}(\tau)$  to denote its image, i.e.,  $\{y \text{ s.t. } \exists x \in A, \tau(x) = y \in B\}$ . We say **assert**  $x$  to mean if  $\neg x$  then output  $\perp$ . Scalars are denoted by  $a$ , vectors by  $\mathbf{a}$ , and formal variables by  $X$ , polynomials in  $\mathbb{Z}_p[X]$  by  $f$ .  $A : B$  denotes list concatenation. We use  $\langle \mathbf{a}, \mathbf{b} \rangle$  to denote inner-product and  $\mathbf{a} \circ \mathbf{b}$  to denote hadamard product.

### 3.1 Idealized models

**Standard Shoup-GGM oracle.** Also known as Random Representation Model [59, 61, 62]. This model provides a natural idealization of computations within cyclic groups of order  $p$ , effectively



abstracting away specific representational details. We consider a generic group model supporting pairing operations [63]. The model is parametrized by  $(p, S_1, S_2, S_t)$ , where  $S_1, S_2$  denote two source groups,  $S_t$  the target group and  $p$  a prime denoting the size of the groups.

At the time of initialization, three random injections  $\tau_i \xleftarrow{\$} \text{INJ}(\mathbb{Z}_p, S_i)$  for  $i \in \{1, 2, t\}$  are picked by the generic group oracle ( $G$  in Fig. 1). All parties then have oracle access to  $G$  and perform all group operations by making relevant queries to  $G$ . Because this model explicitly defines an oracle interface, it becomes a perfect candidate for arguing about UC security, as it naturally allows to observe and extract queries (as also exploited in [13]).

Note that, in this work for our UC security proofs (as in [13]), we will require that protocols use random generators rather than the canonical ones (i.e.,  $\tau_i(1)$ ) provided by  $G$ .

**Figure 1:** Shoup’s GGM oracle:  $G$

|                                                                                   |                                                                   |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------|
| <u>INIT()</u>                                                                     | <u>CANONICALGEN(<math>i</math>)</u>                               |
| 1: For $i \in \{1, 2, t\} : \tau_i \xleftarrow{\$} \text{INJ}(\mathbb{Z}_p, S_i)$ | 1: <b>return</b> $\tau_i(1)$                                      |
| <u>OP(<math>i, (g_1, a_1), (g_2, a_2)</math>)</u>                                 | <u>PAIR(<math>g_1, g_2</math>)</u>                                |
| 1: <b>assert</b> $(g_1, g_2, a_1, a_2) \in S_i^2 \times \mathbb{Z}_p^2$           | 1: <b>assert</b> $(g_1, g_2) \in S_1 \times S_2$                  |
| 2: $h \leftarrow \tau_i(a_1\tau_i^{-1}(g_1) + a_2\tau_i^{-1}(g_2))$               | 2: $h \leftarrow \tau_t(\tau_1^{-1}(g_1) \cdot \tau_2^{-1}(g_2))$ |
| 3: <b>return</b> $h$                                                              | 3: <b>return</b> $h$                                              |

**Symbolic Shoup’s GGM oracle.** We define a symbolic version of  $G$  (denoted by  $s\text{-}G$  in Figure 4). This is needed in our reduction from UC to a stand-alone property. For ease of analysis, we will replace the group functionality in the UC setting with its symbolic counterpart. To successfully reduce to the stand-alone property, we will require that the group functionality in the stand-alone setting is also symbolic. The formulation here is almost the same as that of symbolic group functionality (from [13]) simplified to ignore sessions and observability records.

The differences compared to  $G$  are as follows: Instead of sampling injections,  $\tau_i$ ’s are now lazily sampled. Each fresh group element (that has not been seen before) gets a fresh formal variable  $X$  assigned to it via the TOUCH interface. On receiving a group query, OP for example,  $s\text{-}G$  first computes the polynomial associated with the result ( $f := a_1\tau_i^{-1}(g_1) + a_2\tau_i^{-1}(g_2)$ ), and if  $f$  was never seen before, then samples at random a group encoding for it from the set of remaining encoding (i.e., sampling without replacement). This means that  $\tau$  now is a map between polynomials and group encoding.

This introduces a distinguishing event between  $G$  and  $s\text{-}G$ : for distinct  $(g_1, g_2) \neq (h_1, h_2)$ , it is possible that  $\tau(\tau^{-1}(g_1) + \tau^{-1}(g_2)) = \tau(\tau^{-1}(h_1) + \tau^{-1}(h_2))$ , however this will not happen in  $s\text{-}G$  since the resultant polynomials will be distinct and hence the encoding sampled for them will be distinct. Nonetheless, similar to [13], we can argue that the probability of distinguishing is negligible, meaning that, one can always replace  $G$  with  $s\text{-}G$  and expect security to hold (Lemma 1<sup>7</sup>).

**Standard random oracle.** Random Oracle Model (ROM), idealizes a hash function, and treats it as a truly random mapping where every unique input yields a uniformly random output, independent of any prior queries. Let  $\text{RO} : \langle \text{bit} \rangle \{0, 1\}^* \rightarrow \langle \text{bit} \rangle \{0, 1\}^n$  be a random oracle outputting length- $n$  bit strings.

<sup>7</sup> Although this theorem is in the context of UC and thus considers different sessions and maintains observable tuples, the proof does not need to consider sessions at any point, and only talks about certain properties of polynomials. Hence, the same reasoning applies here as well.

### 3.2 Succinct non interactive zero-knowledge proofs

**Indexed Relation.** An indexed relation  $\mathcal{R}$  is a set of tuples of the form  $(i, x, w)$ , where  $i$  denotes the index,  $x$  the instance, and  $w$  the witness. Typically,  $i$  describes an arithmetic circuit,  $x$  is a public wire assignments,  $w$  denotes the rest of the private inputs, and  $(i, x, w) \in \mathcal{R}$  iff the circuit determined by  $i$  outputs 1 on inputs  $(x, w)$ .

**(Succinct) non-interactive zero knowledge argument of knowledge.** A zkSNARK for an indexed relation  $\mathcal{R}$  is a tuple of PPT algorithms  $\Pi = (Setup, I, P, V)$  that work as follows. All algorithms have oracle access to  $\text{RO}, \text{G}$ ,

- $\text{pp} \leftarrow \text{Gen}(1^\lambda)$ : On input security parameter, it outputs a set of public parameters consisting of group generators. We assume that  $\text{Gen}$  will sample group generators by sampling at random from sets  $S_1, S_2$ , where  $S_1, S_2$  denote the source groups.
- $\text{crs} \leftarrow \text{Setup}(\text{p})$ : On input  $\text{p}$ , it outputs the setup string  $\text{crs}$ .
- $(\text{ipk}, \text{ivk}) \leftarrow I(\text{crs}, i)$ : On input an index  $i$  and  $\text{crs}$ , it deterministically outputs an index-specific proving key  $\text{ipk}$  and a verification key  $\text{ivk}$ .
- $\pi \leftarrow P(\text{ipk}, x, w)$ : On input  $\text{ipk}$ , statement  $x$  and the witness  $w$ , it outputs a proof string  $\pi$ .
- $b \leftarrow V(\text{ivk}, x, \pi)$  On input  $\text{ivk}, x, w$  it outputs a decision bit.

We require a zkSNARK to satisfy completeness, non-interactive zero-knowledge (NIZK), and simulation-extractability (SIMEXT). Completeness means that if a proof string is generated by running the prover’s algorithm honestly, then the verifier accepts except with negligible probability. NIZK intuitively means that a curious verifier does not learn anything from the proof string apart from the fact that the statement is in the language. Formally, this requires the existence of a simulator algorithm that can simulate a proof string by just knowing the statement and some (witness-independent) trapdoor information. Since we use standard definition for these, we present these definitions in [Appendix B](#). Below we describe the syntactic changes in the NIZK definition relevant to our work.

**NIZK.** In this work, we consider NIZK in the *non-programmable* random oracle model. Hence, we give direct access to  $\text{RO}$  in both games. In both experiments, the distinguisher  $\mathcal{A}$  has access to a “prove” and a “verify” interface. As usual, in the “real” game  $\text{NIZK-0}$ , the prove interface is the honest prover algorithm, while in the “ideal” game  $\text{NIZK-1}$  it the simulator’s algorithm ( $S_1$  is simply a wrapper around  $S$ ). The verify interface just implements the honest verifier algorithm in both games. We include this explicit since it will be needed in the reduction from a UC adversary to a NIZK adversary in later sections. We also do not explicitly write an  $S_0$  simulator algorithm that generates the setup string in  $\text{NIZK-1}$ . Instead we assume that the honest setup algorithm already outputs trapdoors that can be used by the simulator to simulate proofs in  $\text{NIZK-1}$ .

**Simulation-Extractability.** This is a stronger notion than the more standard knowledge-soundness definition. It essentially guarantees that proofs are *non-malleable*, i.e., no efficient adversary can maul an existing proof observed during the execution and use it to forge a new proof without knowing the corresponding witness. This is crucial in the context of UC, since without non-malleability the environment  $\mathcal{Z}$  can ask the honest parties/simulator to generate a proof  $\pi$  on an arbitrary  $(x, w)$  pair, and then maul  $\pi$  and submit this for verification. In the real world, the verification would succeed, however, in the ideal world, the simulator/extractor will fail to extract. This would allow  $\mathcal{Z}$  to distinguish between the real and the ideal world experiments.

**Figure 2:** Security game for SIMEXT

| $\text{SIMEXT}_{\mathcal{A}}(1^\lambda)$                                 | $\tilde{V}(x, \pi)$                                                                              |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1: $\text{pp} \leftarrow \text{Gen}(1^\lambda)$                          | 1: $b \leftarrow V^{\text{RO}, \text{G}}(\text{crs}, x, \pi)$                                    |
| 2: $(\text{crs}, \text{td}) \leftarrow \text{Setup}(\text{pp})$          | 2: If $(x, \pi) \in \mathcal{Q}_S$ <b>return</b> 1                                               |
| 3: $\mathcal{Q}_S := \{\}, \text{adWin} = 0$                             | 3: <b>for</b> $i \in \{1, 2, t\}$ <b>do</b> ,                                                    |
| 4: $\text{st} := (\text{td}, \mathcal{Q}_S, \text{adWin})$               | 4: $\{(i, h_j, B_j)\}_{j \in k_i} \leftarrow E_0(\text{crs}, x, \pi)$                            |
| 5: $\mathcal{A}^{\text{RO}, \text{G}, \tilde{P}, \tilde{V}}(\text{crs})$ | 5:   For $j \in k_i$ , $\mathbf{c}^{(i)}[j] \leftarrow \text{FINDREP}(i, h_j, B_j, \text{tr}_G)$ |
| 6: <b>return</b> $\text{adWin}$                                          | 6: <b>end for</b>                                                                                |
| $\tilde{P}(x, w)$                                                        | 7: Run $w \leftarrow E_1(\mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \mathbf{c}^{(t)})$                  |
| 1: The same as $S_1(\cdot)$ (Fig.5)                                      | 8: If $b = 1 \wedge (x, w) \notin \mathcal{R}$ <b>set</b> $\text{adWin} = 1$                     |
|                                                                          | 9: <b>return</b> $b$                                                                             |

**Definition 1.** Let  $\Pi = (\text{Setup}, I, P, V)$  be a non-interactive argument of knowledge for relation  $\mathcal{R}$  satisfying NIZK w.r.t simulator  $S$  (Definition 6).  $\Pi$  satisfies simulation-extractability in the non-programmable random oracle and generic group model w.r.t  $S$ , if there exists an efficient extractor  $E = (E_0, E_1)$  sharing states, such that for all PPT distinguishers  $\mathcal{A}$  the following advantage is negligible in  $\lambda$ ,

$$\text{Adv}_{\mathcal{A}}^{\text{SIMEXT}}(\lambda) := \Pr [\text{SIMEXT}_{\mathcal{A}}(1^\lambda) = 1] \leq \text{negl}(\lambda)$$

where game SIMEXT is defined in Figure 2.

**Differences from standard notions of simulation-extractability.** In our definition, we assume a specific extraction strategy, which we argue is a natural abstraction of how the extractor of several group based zkSNARKs already behaves. We require that the extractor has two sub-algorithms  $E_0, E_1$  such that: (1)  $E_0$  takes the tuple  $(\text{crs}, x, \pi)$  as input and outputs a list of group elements  $h_j$  along with basis  $\{B_j\}$  such that deriving the representation of  $\{h_j\}$  w.r.t to basis  $\{B_j\}$  results in a potential witness; (2) The representation of  $\{h_j\}$  is extracted (denoted by  $\mathbf{c}$ ) by executing the function FINDREP (Algorithm 1) on the group oracle transcript  $\text{tr}_G$ ; (3) finally,  $E_1$  decodes  $\mathbf{c}$  to output the witness.

The function FINDREP simply linearly scans the list of group oracle queries give to it as input and records representation of all group elements w.r.t. some basis  $B$ . Its output is *almost* the full representation. What is missing are representation w.r.t. to any base element not provided as input. In the security proof, one would argue that these elements were not needed for successful extraction. This abstraction especially comes in handy while trying to reduce a UC adversary to a stand-alone one.

The “canonical extractor” we model here is not far from how knowledge extraction already happens in the AGM: the extractor typically gets the group representations directly from the adversary (here we use FINDREP to derive these), and then often applies a trivial decoding function to output the witness (as done by  $E_1$ ). Moreover, in several constructions it is often clear which group elements in the proof string are necessary for extraction. For example, for Bulletproofs the extractor (function  $e$  in Fig.14 in [15]) simply outputs  $a_{I_g}, a_{I_h}$  and  $a_{O_g}$  where  $a_{I_g}$  are the exponents of commitment  $A_I$  that appears in the first round of the proof with respect to basis  $\mathbf{g}$  which are part of the public parameters (similarly for  $a_{I_h}, a_{O_g}$ ). For PIOP based zkSNARKs, the notion of witness-carrying polynomials [4], i.e. the polynomials that encode the entire witness, explicitly marks the commitment value necessary for extraction.

## 4 Preliminaries: Universal Composability

### 4.1 Global random oracle

We consider a hybrid model where all parties have access to a global non-programmable random oracle ( $\mathcal{G}_{\text{RO}}$ ) from [16, 23].  $\mathcal{G}_{\text{RO}}$  can be queried by any party and the environment directly using two interfaces: QUERY and OBSERVE. On QUERY,  $\mathcal{G}_{\text{RO}}$  lazily samples a random value, and otherwise replies by looking up the response in its local table. On OBSERVE, it replies with the set of illegal queries made so far. Each  $\mathcal{G}_{\text{RO}}$  query can be parsed as  $(sid, x)$ . A query  $(sid, x)$  is marked as illegal if it is made by a party in a session  $sid' \neq sid$ . Informally, no honest party ever makes such a query, while a corrupt party or the environment may try to make the simulator/extractor in  $sid$  fail by making all  $\mathcal{G}_{\text{RO}}$  queries outside the session. However, in the latter case, the simulator/extractor will now have access to these queries via OBSERVE. The description of the standard  $\mathcal{G}_{\text{RO}}$  functionality can be found in Appendix C.

Note that, [26] and [22] require a programmable  $\mathcal{G}_{\text{RO}}$  since they consider IOP based zkSNARKs which requires programming for showing zero-knowledge. We only require observable  $\mathcal{G}_{\text{RO}}$  as we focus on PIOP based zkSNARKs such as Plonk and Marlin, which are compiled using variants of KZG [46] polynomial commitments. The zero-knowledge simulator in this case simulates via knowing the trapdoor rather than programming.

### 4.2 Global generic group functionality with restricted observability

We show UC security in the presence of the global generic group functionality (Functionality 5) from [13]. In our proof, in Section 5, we will primarily use the symbolic version of  $\mathcal{G}$ . Hence, here we discuss  $\mathcal{G}$  informally, moving the formal description to Appendix C.

In addition to group operations as in Figure 1,  $\mathcal{G}$  must manage observability. The observability rules are defined via two rules  $\text{Legal}_{sid}^{\text{OP}}$  and  $\text{Legal}_{sid}^{\text{PAIR}}$ . Each group element seen during the execution is maintained as a formal variable or a polynomial in these variables. Whenever a query is made with a group element  $g$  in a session  $sid$  that has not been seen before, the functionality registers it by assigning a fresh variable for it (via TOUCH), and stores it in  $R_i[g]$ .  $g$  will essentially be treated as a generator for session  $sid$ . This is used by (honest) protocols to generate random generators for their execution session.

When an  $\text{OP}_{sid}$  query is made, the functionality computes and adds the polynomial corresponding to the result to  $R_i[h]$  (for example,  $R_i[h] \cup a_1 R_i[g_1] + a_2 R_i[g_2]$ ). Then, it checks if  $\exists f \in R_i[h]$  such that  $f$  contains any variable from a session  $sid' \neq sid$  or a non-zero constant term (membership in  $\text{Legal}_{sid}^{\text{OP}}$ ). If this is the case, then the query is added to the observable set. This enforces the following behaviour: to not be observable a party in session  $sid$  must operate only on the group elements that are derived from the group generators in  $sid$ . Group operations with the canonical generator (which results in a constant term) are also considered illegal.

When a  $\text{PAIR}_{sid}$  query is made, the functionality checks the membership of  $f$  in  $\text{Legal}_{sid}^{\text{PAIR}}$  where in addition to the conditions in  $\text{Legal}_{sid}^{\text{OP}}$  it also checks that any degree 1 term in  $f$  only comes from the target group variables. This rules out pairing operations with the canonical generator. [13] considers these operations as legal. However, the choice is somewhat arbitrary. We decide to add this constraint as it enforces a behaviour that is closer to the expected honest behaviour, i.e., honest parties only operate on group elements that are derived from the group generators in  $sid$ .

### 4.3 Global symbolic group functionality with perfect session separation

As explained in [13], an issue with  $\mathcal{G}$  is that it has imperfect domain separation, i.e., some group elements may belong to two sessions. Ideally, this should amount to an error event in practice. To

resolve this, [13] proposes a purely symbolic version of  $\mathcal{G}$  (denoted by  $\mathbf{s}\mathcal{G}$  in this work, Functionality 1), which admits perfect session separation.

The main differences between the two functionalities are in how the map  $\tau$  is defined and in the enforcement of domain separation to define observability. In  $\mathcal{G}$ , the internal maps  $\tau_i$  assigns random group elements to scalar values in  $\mathbb{Z}_p$ , and use a table  $R_i$  to keep track of every possible representation of a group element to determine observability. In contrast, the Symbolic Global Generic Group functionality ( $\mathbf{s}\mathcal{G}$ ) modifies the domain of  $\tau_i$  to map formal polynomials in  $\mathbb{Z}_p[X_{sid}, \dots]$  directly to group elements. Since the number of symbolic polynomials is unbounded, it is impossible to take a random injection. Instead,  $\tau_i$  are lazily sampled in  $S_i \setminus \text{img}(\tau_i)$  (i.e. without replacement). Considering polynomial time adversaries, the total amount of queries are  $\mathcal{O}(\text{poly}(\lambda)) \ll |S|$  and thus the functionality never runs out of elements. By assigning unique formal variables to generators,  $\mathbf{s}\mathcal{G}$  enforces a different representation for *all* the elements and therefore allows perfect equality check using the encoding:

$$\tau_i(f(X \dots)) = \tau_i(f'(X \dots)) \iff f(X \dots) = f'(X \dots)$$

Looking ahead, using a symbolic functionality simplifies the analysis for reducing UC to stand-alone properties since the above equality check ensures that the adversary built in the reduction behaves identically to  $\mathbf{s}\mathcal{G}$  would in the hybrid games.

We use a simplified version of  $\mathbf{s}\mathcal{G}$ . In particular, we do not require the extra internal interfaces available to the simulation (e.g., COMPUTESYMBOLIC, etc), or Laurent polynomials. This is because, we reduce to the underlying stand-alone property in a black-box manner (instead of showing an explicit proof of simulation-extractability as is shown in [13] to argue indistinguishability between hybrids  $H_1$  and  $H_2$  in Claim 5). Because if this, we do not need to track the internal computation of the simulator or it's randomness (the latter requires Laurent polynomials).

It was shown in [13] that replacing  $\mathcal{G}$  with its symbolic counterpart  $\mathbf{s}\mathcal{G}$  is indistinguishable to all algorithms making up to  $Q$  oracle queries.

**Lemma 1** ([13]). *Let  $\mathcal{O}_{real} = \mathcal{G}[\text{CANONICALGEN}, \text{OBSERVE}, \text{TOUCH}, \text{OP}, \text{PAIR}]$ . Let  $\mathcal{O}_{sym} = \mathbf{s}\mathcal{G}[\text{CANONICALGEN}, \text{OBSERVE}, \text{TOUCH}, \text{OP}, \text{PAIR}]$ . For all algorithms  $\mathcal{B}$  that make at most  $Q$  oracle queries, it holds that:*

$$\left| \Pr[\mathcal{B}^{\mathcal{O}_{real}} = 1] - \Pr[\mathcal{B}^{\mathcal{O}_{sym}} = 1] \right| \leq \binom{3Q+1}{2} \cdot 2/p \leq (9Q^2 + 3Q)/p$$

*When treating interfaces `INTERFACE` as oracles, this means that the caller specifies session  $sid$  and input  $x$ , then gets the result of `INTERFACEsid(x)`. The oracles share state.*

#### 4.4 NIZK and setup functionality

We use syntactic variant of the standard  $\mathcal{F}_{\text{nizk}}$  functionality [13, 44], and a setup functionality  $\mathcal{F}_{\text{setup}}$  to model trusted generation of the setup string (crs) for the non-interactive proof system. The  $\mathcal{F}_{\text{setup}}$  can be found in Appendix C. Informally, it has two interfaces: (1) `INITsid` is invoked at the time of initialization. It first generates the group generators for each source group, by sampling at random from sets  $S_i$  and registering them with  $\mathcal{G}$  via `TOUCH`. It then generates `crs` by running `Setup` using these generators. (2) `GETCRSsid` can be called by parties in session  $sid$  to obtain `crs`.



### Functionality 1: $s\text{-}\mathcal{G}$

Parameters of the functionality:

- A prime number  $p$
- Sets  $S_1, S_2, S_t$  with  $|S_i| = p$  for all  $i \in \{1, 2, t\}$ .

Internal state maintained by the functionality:

- $\tau_i$ : Maps polynomials in  $\mathbb{Z}_p[X_{sid}, \dots]$  (instead of  $\mathbb{Z}_p$ ) to  $S_i$ , for  $i \in \{1, 2, t\}$ .
- $X_{i,sid}$ : Initially empty list. This will record formal variables defined for session  $sid$ .  $X_{sid}$  denotes the concatenation  $X_{1,sid} : X_{2,sid} : X_{t,sid}$ .
- $Ob$ : Initially empty list. This will record the set of observable group operation queries.

Furthermore:

- A polynomial  $f$  is in  $\text{Legal}_{sid}^{\text{OP}}$  if it only contains variables in  $X_{sid}$  and a zero constant term.
- A polynomial  $f$  is in  $\text{Legal}_{sid}^{\text{PAIR}}$  if in addition to constraints in  $\text{Legal}_{sid}^{\text{OP}}$ , for all degree 1 monomials in  $f$  of the form  $c \cdot X$ , it holds that  $X \in X_{t,sid}$ .

INIT<sub>sid</sub>()

```
1: for  $i \in \{1, 2, t\}$  : do
2:    $\tau_i \leftarrow \{\}$ 
3:   TAU( $i, 1$ )
4: end for
```

CANONICALGEN<sub>sid</sub>( $i$ )

```
1: return TAU( $i, 1$ )
```

OBSERVE<sub>sid</sub>

```
1: return  $Ob$ 
```

OP<sub>sid</sub>( $i, (g_1, a_1), (g_2, a_2)$ )

```
1: assert  $(g_1, g_2, a_1, a_2) \in S_i^2 \times \mathbb{Z}_p^2$ 
2: for  $j \in \{1, 2\}$  do
3:   TOUCHsid( $i, g_j$ )
4: end for
5:  $f \leftarrow a_1 \tau_i^{-1}(g_1) + a_2 \tau_i^{-1}(g_2)$ 
6:  $h \leftarrow \text{TAU}(i, f)$ 
7: if  $f \notin \text{Legal}_{sid}$  then
8:    $Ob \leftarrow Ob \cup \{(OP, i, g_1, g_2, a_1, a_2, h)\}$ 
9: end if
10: return  $h$ 
```

TOUCH<sub>sid</sub>( $i, g$ )

```
1: if  $g \notin \text{Im}(f)$  then
2:   Initialize fresh variable  $X$ 
3:    $X_{i,sid} \leftarrow X_{i,sid} : [X]$ 
4:    $\tau_i(X) \leftarrow g$ 
5: end if
```

TAU( $i, f$ )

```
1: if  $\tau_i(f) = \perp$  then
2:    $\tau_i(f) \xleftarrow{\$} S_i \setminus \text{Im}(\tau_i)$ 
3: end if
4: return  $\tau_i(f)$ 
```

PAIR<sub>sid</sub>( $g_1, g_2$ )

```
1: assert  $(g_1, g_2) \in S_1 \times S_2$ 
2: for  $i \in \{1, 2\}$  do
3:   TOUCHsid( $i, g_i$ )
4: end for
5:  $f \leftarrow \tau_t(\tau_1^{-1}(g_1) \cdot \tau_2^{-1}(g_2))$ 
6:  $h \leftarrow \text{TAU}(t, f)$ 
7: if  $f \notin \text{Legal}_{sid}$  then
8:    $Ob \leftarrow Ob : [(PAIR, g_1, g_2, h)]$ 
9: end if
10: return  $h$ 
```

**Functionality 2:  $\mathcal{F}_{\text{nizk}}$**

INIT<sub>sid</sub>( )

1:  $T \leftarrow []$

PROVE<sub>sid</sub>( $x, w$ )

1: **assert**( $x, w$ )  $\in \mathcal{R}$

2:  $\pi \leftarrow \mathcal{S}.\text{SIMULATE}_{\text{sid}}(x)$

3:  $T \leftarrow T \cup (x, \pi)$

4: **return**  $\pi$

VERIFY<sub>sid</sub>( $x, \pi$ )

1: **if** ( $(x, \pi) \in T$ ) **then return** 1

2: **end if**

3:  $w \leftarrow \mathcal{S}.\text{EXTRACT}_{\text{sid}}(x, \pi)$

4: **if** ( $(x, w) \in \mathcal{R}$ ) **then return** 1

5: **end if**

6: **return** 0

#### 4.5 UC security in the $\mathcal{G}$ -hybrid

We recall the standard syntax for ideal functionalities and protocols in the UC framework in [Appendix C](#).

The UC security of protocol  $\Pi$  against static corruptions is formally defined in [Definition 2](#) and consist of showing that no efficient environment who can corrupt parties statically can distinguish between the real and the ideal world.

**Definition 2.** We say that protocol  $\Pi$  UC-realizes the functionality  $\mathcal{F}_{\text{nizk}}$  in the  $(\mathcal{G}, \mathcal{F}_{\text{setup}})$ -hybrid model if for all PPT adversaries  $\mathcal{A}$  there exists a PPT simulator  $\mathcal{S}$  such that for all PPT environments  $\mathcal{Z}$ , the following ensembles are indistinguishable:

$$\left\{ \text{REAL}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{G}, \mathcal{F}_{\text{setup}}}(\lambda, z) \right\}_{\lambda \in \mathbb{N}, z \in \{0,1\}^*} \approx \left\{ \text{IDEAL}_{\mathcal{F}_{\text{nizk}}, \mathcal{S}, \mathcal{Z}}^{\mathcal{G}}(\lambda, z) \right\}_{\lambda \in \mathbb{N}, z \in \{0,1\}^*}$$

### 5 UC security from Game-Based Definitions

In this section, we present a general lifting theorem that bridges security definitions in the stand-alone setting with UC framework. We show that any zkSNARK satisfying completeness, zero-knowledge, and simulation-extractability in Shoup's GGM+non-programmable ROM is UC-secure in the global-GGM+non-programmable GROM hybrid model.

#### 5.1 Stand-alone protocol to UC-secure protocol

Let  $\text{Gen}$  be the algorithm producing public group parameters  $\text{pp}$  given the description of the group, and let  $\Pi = (\text{Setup}, I, P, V)$  be a non-interactive argument of knowledge in the presence of G+non-programmable ROM. A natural transformation of  $\Pi$ , denoted by  $\mathbf{\Pi} = (\mathbf{Setup}, \mathbf{I}, \mathbf{P}, \mathbf{V})$  given below is the corresponding protocol in the UC-framework in the  $\mathcal{F}_{\text{setup}}, \mathcal{G}, \mathcal{G}_{\text{RO}}$ -hybrid. This is just a wrapper around the stand-alone algorithms with the change being that whenever the underlying generic algorithm makes a call to a GGM oracle, it is forwarded to the  $\mathcal{G}$  functionality, and when a random oracle query is made, it is forwarded to  $\mathcal{G}_{\text{RO}}$ . We assume that the index can be parsed from the statement and that the prover and the verifier run the indexer algorithm internally to obtain the index and then run the algorithms from  $\Pi$ . This is merely for convenience and to not deviate from the main focus of this work. One could define a version of  $\mathcal{F}_{\text{nizk}}$  with an indexer interface so that preprocessing based snarks can semantically fit in the UC framework.

**Protocol 1:  $\Pi$**

**Setup<sub>sid</sub>**

1: Call  $\mathcal{F}_{\text{setup}}.\text{INIT}()$

**P<sub>sid</sub>( $x, w$ )**

1: **if**  $(x, w) \notin \mathcal{R}$  **then return**  $\perp$   
 2: **end if**  
 3:  $\text{crs} \leftarrow \mathcal{F}_{\text{setup}}.\text{GETCRS}_{\text{sid}}()$   
 4: Parse  $x$  to obtain  $i$   
 5:  $(\text{ipk}, \text{ivk}) \leftarrow I(\text{crs}, i)$   
 6:  $\pi \leftarrow P^{\mathcal{G}, \mathcal{G}_{\text{RO}}}(\text{ipk}, x, w)$   
 7: **return**  $\pi$

**V<sub>sid</sub>( $x, \pi$ )**

1:  $\text{crs} \leftarrow \mathcal{F}_{\text{setup}}.\text{GETCRS}_{\text{sid}}()$   
 2: Parse  $x$  to obtain  $i$   
 3:  $(\text{ipk}, \text{ivk}) \leftarrow I(\text{crs}, i)$   
 4:  $b \leftarrow V^{\mathcal{G}, \mathcal{G}_{\text{RO}}}(\text{ivk}, x, \pi)$   
 5: **return**  $b$

**Theorem 4.** Let  $\Pi = (\text{Setup}, P, V)$  be a SNARK satisfying completeness (Definition 5), non-interactive zero-knowledge (Definition 6) and simulation-extractability (Definition 1) in Shoup’s GGM (Figure 1) and the random oracle model. Then the protocol  $\Pi = (\text{Setup}, P, V)$  in Protocol 1 UC-realises  $\mathcal{F}_{\text{nizk}}$  in the  $\mathcal{F}_{\text{setup}}, \mathcal{G}, \mathcal{G}_{\text{RO}}$ -hybrid model.

*Proof. Simulator definition.* We give the simulator construction  $\mathcal{S}$  in Algorithm 2. Let  $(E, S)$  be the extractor and simulator algorithms for  $\Pi$ .  $\mathcal{S}$  acts as a wrapper for group and random oracle queries made by the adversary and maintains the list  $Ob_{\mathcal{S}}$  which consists of all queries made in this session that are directly observed by  $\mathcal{S}$ . It is responsible for generating the view of  $\mathcal{F}_{\text{setup}}$  and hence knows the trapdoors associated with  $\text{crs}$ . For simulating proofs on PROVE command, it internally runs  $S$  given the  $\text{crs}$  trapdoor. Finally, to respond to VERIFY command, it runs the extraction algorithm  $E$ . It first completes its observation list by making an  $\text{OBSERVE}_{\text{sid}}$  query and concatenating it with the local set  $Ob_{\mathcal{S}}$ , and then it runs the canonical extractors  $E_0, E_1$  for the concatenated oracle transcript.

**Hybrids.** Next, we describe a series of hybrids from Real to Ideal experiments. We refer to Section 5.2 for the description of the adversary we build in the reduction, and Section 5.3 for indistinguishability claims.

*A reminder about notations.* Without loss of generality, we consider only two sessions: the challenge session  $\text{sid}$  and the other session  $\text{sid}'$ . We assume that  $\mathcal{Z}$  issues INIT, PROVE, VERIFY commands only for the challenge session  $\text{sid}$ , while may use  $\text{sid}'$  to query  $\mathcal{G}, \mathcal{G}_{\text{RO}}$ . This is again w.l.o.g since commands in  $\text{sid}'$  can be reduced to making several equivalent group queries (and other local non-group operations). Below we will separate OP, PAIR operations for  $\text{sid}, \text{sid}'$  as they behave slightly differently. All other group operations will be handled identically for both  $\text{sid}, \text{sid}'$  (hence we only write explicitly for  $\text{sid}$ ). To distinguish the group we explicitly mention in the subscript as  $X_{i, \text{sid}}$ . When not necessary, we ignore this subscript. We also ignore the subscript of map  $\tau$  when it is clear from the context. We do not specify exact source group for pairings, rather express it as a Hadamard product to be interpreted as:  $X_{\text{sid}} \circ X_{\text{sid}'} = X_{1, \text{sid}} \circ X_{2, \text{sid}'} + X_{2, \text{sid}} \circ X_{1, \text{sid}'}$ .

We assume that the environment makes  $s_1$  PROVE commands and  $s_2$  VERIFY commands.

Note that hybrids  $\text{Hyb}_4\text{-Hyb}_8$  mainly have syntactic differences between them. We prefer to write them explicitly in the hybrids to bring it closer to the reduction adversary’s description. This way, most of the argument for indistinguishable view generated by the adversary in the reduction can be argued here, making it easier to parse the reduction. We consider the following hybrids:

- $\text{Hyb}_0$ : Real world security game (as defined in Definition 2).
- $\text{Hyb}_1$ : Replace  $\mathcal{G}$  with its symbolic counter-part.

- **Hyb<sub>2</sub>**: Replace all the honest parties with a single party  $\mathcal{B}$ . This party is responsible for simulating the view of the adversary and the environment for the rest of the protocol.  $\mathcal{B}$  acts on behalf of honest parties doing exactly what an honest party does in the real world experiment. In addition, it intercepts queries to  $\mathcal{G}, \mathcal{G}_{\text{RO}}$  relays back the responses (using interfaces  $\text{GETCRS}_{\text{sid}}, \dots$  in [Algorithm 2](#)). Similarly, it intercepts queries for  $\mathcal{F}_{\text{setup}}$  and relays back the response.
- **Hyb<sub>3</sub>**: In this hybrid we introduce an additional abort condition corresponding to the case where  $\mathcal{Z}$  makes a group query using a element  $g$  such that  $g$  appears as an intermediate group element in an honest algorithm's execution. Define  $\text{Out}_H^{(t)}$  as the list of all group elements obtained as a result of group queries made by honest algorithm (all outputs  $h$  received by  $\mathcal{B}$  and  $\mathcal{F}_{\text{setup}}$  in  $\text{sid}$ ),  $\text{Query}_{\mathcal{Z}}^{(t)}$  as the list of all group queries made by  $\mathcal{Z}$  in all sessions (this includes queries in  $\text{sid}'$  and queries made by the corrupt party  $\mathcal{A}$  in  $\text{sid}$ ), and  $\text{View}_{\mathcal{Z}}^{(t)}$  as list of group elements seen by  $\mathcal{Z}$  (these includes outputs of honest algorithm executions by  $\mathcal{B}$  and  $\mathcal{F}_{\text{setup}}$ , and outputs of group queries made by  $\mathcal{Z}$ ) up until time  $t$ . Let  $\text{Query}_{\mathcal{Z}}^{(t)}$  be ordered according to the time when queries were made. In this hybrid, we abort if: let  $g$  be a group element that appears in a group query made by  $\mathcal{Z}$  for the first time at time  $t$ , i.e., at time  $t$   $\mathcal{Z}$  makes a linear query of the form  $((i, g, \cdot, \cdot, \cdot))$  or  $(i, \cdot, g, \cdot, \cdot)$  or a pair query  $((g, \cdot)$  or  $(\cdot, g))$ , such that  $g \in \text{Out}_H^{(t)} \setminus \text{View}_{\mathcal{Z}}^{(t)}$ .
- **Hyb<sub>4</sub>**: Change behaviour of  $\mathbf{s}\mathcal{G}$  for observability: On deriving the resultant polynomial in OP, PAIR queries, if the constant term is non zero then add it to  $Ob$  (i.e., even without checking the full polynomial). For PAIR queries also check that the degree 1 monomials with a variable  $X$  are such that  $X \in X_{t, \text{sid}}$ .
- **Hyb<sub>5</sub>**: Change behaviour of  $\mathbf{s}\mathcal{G}$  so that it decides observability for session  $\text{sid}$  for linear queries as: Let  $h \leftarrow \text{OP}_{\text{sid}}(i, g_1, g_2, a_1, a_2)$  be an OP query. Compute the polynomial corresponding to  $h$  as before, i.e.,  $f := a_1 \tau_i^{-1}(g_1) + a_2 \tau_i^{-1}(g_2)$ . Now, partition the polynomial according to  $\text{sid}$  and  $\text{sid}'$  parts: rewrite  $f := f^{(1)}(X_{i, \text{sid}}) + f^{(2)}(X_{i, \text{sid}'})$ , where both polynomials are linear with zero constant term (the constant term will be zero because of the previous hybrid). Instead of checking legality of the full polynomial  $f$ , only check if  $f^{(2)}(X_{i, \text{sid}'})$  is a zero polynomial. If not, add this tuple to  $Ob$ .
- **Hyb<sub>6</sub>**: Change behaviour of  $\mathbf{s}\mathcal{G}$  so that it decides observability for session  $\text{sid}'$  for linear queries as: Rewrite  $f := f^{(1)}(X_{i, \text{sid}}) + f^{(2)}(X_{i, \text{sid}'})$  as in the previous hybrid. Only check if  $f^{(1)}(X_{i, \text{sid}'})$  is a zero polynomial. If not, add this tuple to  $Ob$ .
- **Hyb<sub>7</sub>**: Change behaviour of  $\mathbf{s}\mathcal{G}$  for pairing queries in  $\text{sid}$ : Let  $h \leftarrow \text{PAIR}_{\text{sid}}(g_1, g_2)$  be a PAIR query. Compute the polynomial corresponding to  $h$  as before, i.e.,  $f := \tau_1^{-1}(g_1) \cdot \tau_2^{-1}(g_2)$ . Rewrite  $f := f^{(1)}(X_{\text{sid}}) + f^{(2)}(X_{\text{sid}}, X_{\text{sid}'}),$  where  $f^{(2)} := \langle v_1, X_{\text{sid}} \circ X_{\text{sid}'} \rangle + \langle v_2, X_{\text{sid}'} \circ X_{\text{sid}'} \rangle$  for some vector of field elements  $v_1, v_2$ . Do the following checks and add the tuple to  $Ob$  if either check fails.
  - Rewrite term  $\langle v_1, X_{\text{sid}} \circ X_{\text{sid}'} \rangle$  as  $f^{(3)}(X_{\text{sid}'}) \cdot f^{(4)}(X_{\text{sid}})$ . Check if the product polynomial is a zero polynomial.
  - Check that  $\langle v_2, X_{\text{sid}'} \circ X_{\text{sid}'} \rangle$  is a zero polynomial.
- **Hyb<sub>8</sub>**: Change behaviour of  $\mathbf{s}\mathcal{G}$  for pairing queries in  $\text{sid}'$ : Partition  $f$  as in the previous hybrid. Check if  $f^{(1)} + \langle v_1, X_{\text{sid}} \circ X_{\text{sid}'} \rangle$  is a zero polynomial, and if not add this tuple to  $Ob$ . To check if the polynomial is zero do the following:
  - Check if  $f^{(1)}$  is a zero polynomial.
  - Check term  $\langle v_1, X_{\text{sid}} \circ X_{\text{sid}'} \rangle$  as in the previous hybrid.
- **Hyb<sub>9</sub>**: Instead of forwarding calls to  $\mathcal{F}_{\text{setup}}$ , run  $\text{INIT}_{\text{sid}}()$  and  $\text{GETCRS}_{\text{sid}}$  code in the simulator  $\mathcal{S}.\text{INIT}_{\text{sid}}$  and  $\mathcal{S}.\text{GETCRS}_{\text{sid}}$  ([Algorithm 2](#)) to respond to the queries.

- For  $j \in [s_1]$ ,  $\text{Hyb}_{9+j}$ : For the  $j$ -th  $\text{PROVE}(x, w)$  command issued by  $\mathcal{Z}$  for an honest party, replace the honest prover's algorithm with  $\mathcal{S}.\text{SIMULATE}_{sid}(x)$  (as described in Algorithm 2).
- For  $j \in [s_2]$ ,  $\text{Hyb}_{9+s_1+j}$ : For the  $j$ -th  $\text{VERIFY}(x, \pi)$  command issued by  $\mathcal{Z}$  for an honest verifier, replace the honest verifier's algorithm with  $\mathcal{S}.\text{EXTRACT}_{sid}(x, \pi)$  (as described in Algorithm 2).
- $\text{Hyb}_{9+s_1+s_2}$ : Change  $\mathbf{s}\text{-}\mathcal{G}$  behaviour for deciding observability to as it is decided in  $\text{Hyb}_1$ .
- $\text{Hyb}_{10+s_1+s_2}$ : Replace  $\mathbf{s}\text{-}\mathcal{G}$  with its non-symbolic counterpart.
- $\text{Hyb}_{11+s_1+s_2}$ : Replace  $\mathcal{B}$  by  $\mathcal{S}$ . This is the ideal world.

## 5.2 Adversary's definition

Before arguing indistinguishability between each hybrid, we informally describe the adversary  $\mathcal{A}^*$  against some property based definition with interfaces  $(\text{Setup}, P, V)$ . Given black-box access to  $\mathcal{Z}$  in the hybrids,  $\mathcal{A}^*$  breaks a property of the underlying zkSNARK. The adversary's algorithm remains the same in each hop, hence we define it only once. The full formal description can be found in Appendix D.

The overall reduction strategy is as follows: Let  $\mathcal{Z}$  be a UC adversary that can successfully distinguish between two consecutive hybrids  $\text{Hyb}_i, \text{Hyb}_{i+1}$ .  $\mathcal{A}^*$  is given black-box access to  $\mathcal{Z}$  and is responsible for generating indistinguishable view for  $\mathcal{Z}$  as in the hybrid games while interacting with the challenger of an underlying property based security notion and with access to a symbolic Shoup's GGM oracle  $\mathbf{s}\text{-}\mathcal{G}$  (Figure 4) and a standard random oracle  $\text{RO}$ .  $\mathcal{A}^*$  will act as a wrapper around  $\mathcal{Z}$  and forward any  $\mathcal{G}_{\text{RO}}$  query with  $sid$  prefix to  $\text{RO}$ , while locally simulating the responses for  $sid'$  via lazy sampling. When  $\mathcal{Z}$  asks for  $\mathcal{G}_{\text{RO}}.\text{OBSERVE}$ ,  $\mathcal{A}^*$  can correctly respond to this since it sees all queries made by  $\mathcal{Z}$  and hence it can maintain the set of illegal queries. All  $\mathbf{s}\text{-}\mathcal{G}$  queries, regardless of the session will be forwarded to  $\mathbf{s}\text{-}\mathcal{G}$ . To be able to respond to  $\mathcal{G}_{\text{RO}}.\text{OBSERVE}$ ,  $\mathcal{A}^*$  can additionally maintain its own version of tables  $\tau$  by registering each new group element with a fresh variable. This exactly matches with the tables maintained by  $\mathbf{s}\text{-}\mathcal{G}$  (apart from variables having different representations).

When an  $\text{INIT}_{sid}$  command is received,  $\mathcal{A}^*$  interacts with the challenger ( $Ch$ ) of underlying game to receive the  $\text{crs}$  for current execution. Later, when a prove or a verify query is made by  $\mathcal{Z}$ ,  $\mathcal{A}^*$  passes these on to the prove/verify interfaces of the underlying security game. These commands cause an issue with maintenance of the group observability sets. Unlike the real UC experiment,  $\mathcal{A}^*$  only have partial access to the groups representations. More specifically, it doesn't have access to the representation for any internal group element computed by the algorithms of  $Ch$ .

However, the reduction relies on the observation that the adversary does not need the full polynomial representation of honest elements to distinguish legal from illegal queries. It only requires the ability to verify if a query cancels out the "foreign" variables associated with other sessions  $sid'$ . Concretely,  $\mathcal{A}^*$  decomposes operations into components depending on known adversarial variables and unknown honest proof variables. It then performs local checks via group oracle queries to determine which components of the polynomial vanishes, thereby correctly identifying illegal queries. For example, let  $\pi$  be a group element in the output of a  $\text{PROVE}$  query. Since  $\mathcal{A}^*$  does not know how it was computed, it just assigns a fresh variable  $X_\pi$  for it. Later, assume that  $\mathcal{Z}$  makes an  $\text{OP}_{sid'}(i, g_1, g_2, a_1, a_2)$  query such that  $g_1$  can be decomposed by  $\mathcal{A}^*$  as a polynomial  $a \cdot X_{sid} + b \cdot X_\pi$ , and  $g_2$  as  $Y_{sid'}$ . Let  $g$  be the group element corresponding to the variable  $X_{sid}$ . To correctly decide observability for session  $sid'$ ,  $\mathcal{A}^*$  needs to check that all terms corresponding to  $sid$  must be 0. It can do this by checking if  $\mathbf{s}\text{-}\mathcal{G}.\text{OP}(i, g, \pi, a, b)$  returns a value corresponding to 0 (i.e. the result is equal to the encoding for 0).



Completing the informal description of the adversary: to determine observability,  $\mathcal{A}^*$  decomposes the resultant polynomials exactly as specified in hybrids  $\text{Hyb}_4$ - $\text{Hyb}_8$  but in terms of variables it stores locally. Instead of checking if some specified term of polynomial is non-zero (as is done in each hybrid), it perform group operations corresponding to this term and checks if the result is an encoding of 0.

### 5.3 Indistinguishability between hybrids

**Notations.** Recall that  $\mathbf{s}\text{-G}$  refers to the symbolic version of shoup's GGM. For common data-structures (for example  $\tau$ ) that would be maintained both by the adversary and  $\mathbf{s}\text{-G}$ , we denote the former simply as  $\tau$ , while  $\mathbf{s}\text{-G}.\tau$  for the latter. We use,  $X_\pi$  denote the set of variables associated with group elements that are output by either the  $\text{PROVE}$  command or  $\text{INIT}_{sid}$  execution.  $X_{sid}$  and  $X_\pi$  together contain all variables belonging to session  $sid$ . When we write  $f^{(3)}(\tau(X_{sid'}))$ , it is a short hand to mean that each  $X$  in  $f$  should be replaced by corresponding group element, and sequence of group operation should be performed according to the definition of  $f$ .

Indistinguishability for  $\text{Hyb}_0 - \text{Hyb}_1$  is given by Lemma 3 in [13].  $\text{Hyb}_1$ - $\text{Hyb}_9$  differ mainly syntactically and hence can be shown to be indistinguishable. We move the corresponding claims and proofs to [Appendix E](#). Below we focus on the reduction to NIZK and SIMEXT for showing indistinguishability for hybrids  $\text{Hyb}_{9+j}$  to  $\text{Hyb}_{10+s_1+s_2}$ .

**Claim 1.** For  $j \in \{0, \dots, s_1 - 1\}$ ,  $\text{Hyb}_{9+j}$  and  $\text{Hyb}_{9+j+1}$  are indistinguishable.

*Proof.* The difference in  $\text{Hyb}_{9+j+1}$  is that it runs  $\text{SIMULATE}$  instead of prover's algorithm to generate proof string. Let  $\mathcal{Z}$  be the adversary that distinguishes between the two hybrids. Then given  $\mathcal{Z}$ , we can construct an adversary  $\mathcal{A}_{\text{NIZK}}$  against NIZK property of  $\Pi$  ([Definition 6](#)). The code of  $\mathcal{A}_{\text{NIZK}}$  is the same that of  $\mathcal{A}^*$  described in [Section 5](#) and [Appendix D](#).

If we establish: when  $b = 0$  in the NIZK game, the view generated by  $\mathcal{A}_{\text{NIZK}}$  is indistinguishable from  $\mathcal{Z}$ 's view in  $\text{Hyb}_{9+j}$ ; and when  $b = 1$ , it is indistinguishable from the view in  $\text{Hyb}_{9+j+1}$ . Then given this,  $\mathcal{Z}$ 's distinguishing advantage directly translates to  $\mathcal{A}_{\text{NIZK}}$ 's advantage in the NIZK game. In the rest of the proof we argue the indistinguishability of the view generated by  $\mathcal{A}_{\text{NIZK}}$ .

Consider the view generated for  $b = 0$  (the same argument holds for  $b = 1$ ): It is clear from observation that when  $\mathcal{Z}$  does not make any  $\text{INIT}_{sid}$  or  $\text{PROVE}_{sid}$  command then the view generated by  $\mathcal{A}_{\text{NIZK}}$  is identical to the one in  $\text{Hyb}_{9+j}$  (since all group operations would be observed by  $\mathcal{A}_{\text{NIZK}}$ ). When an  $\text{INIT}_{sid}$  or a  $\text{PROVE}_{sid}$  command is made the group operations associated with these computations are performed by the  $(\text{Setup}, P)$  algorithms within the NIZK game. Hence, the trace of intermediate group operations is not available to  $\mathcal{A}^*$ . Instead,  $\mathcal{A}^*$  assigns fresh variables for any group element present in the output of  $(\text{INIT}_{sid}, \text{PROVE}_{sid})$  queries and adds them to the set  $X_\pi$ . On subsequent group queries  $\mathcal{A}^*$  decides whether or not to add the group operation tuple to  $Ob$  using  $X_\pi$  (instead of the full representation in terms of  $X$  that would be available in  $\text{Hyb}_{9+j}$ ). This may possibly lead to misclassification of group operation tuples: tuples marked as illegal in  $\text{Hyb}_{9+j}$  might not be marked as such  $\mathcal{A}^*$ , and vice versa. This may happen only in the two following cases:

**Case 1: Collisions with Internal Variables.** If  $\mathcal{Z}$  makes a group query containing a group element  $g$  such that  $g$  has not been seen before by  $\mathcal{A}_{\text{NIZK}}$  but was a part of an intermediate group operation by  $(\text{Setup}, P)$  computation. In  $\text{Hyb}_{9+j}$ , the group functionality would define the polynomial representation of  $g$  with respect to  $sid$  variables. However, in the view generated by  $\mathcal{A}_{\text{NIZK}}$ ,  $g$  would be defined w.r.t the session in which the query first appears. This could be in  $sid' \neq sid$ . Hence, the contents of  $Ob$  may now differ which can be observed by  $\mathcal{Z}$  by calling  $\text{OBSERVE}$ . This case is however already ruled out in  $\text{Hyb}_3$ .

**Case 2: Algebraic Dependencies.** We now consider queries where  $\mathcal{Z}$  combines a group element present in the output of  $\text{INIT}_{sid}$ ,  $\text{PROVE}_{sid}$  query with other group elements and  $\mathcal{A}^*$  must decide whether the operation is observable or not. While we concretely refer to  $\text{PROVE}_{sid}$  below, as before the argument for  $\text{INIT}_{sid}$  and  $\text{PROVE}_{sid}$  is the same.

**Linear Operations in  $sid$ :**  $h \leftarrow \text{OP}_{sid}(i, (g_1, a_1), (g_2, a_2))$ .

The bulk of the argument below is that the polynomial representation for  $h$  defined by  $\mathbf{s}\mathcal{G}$  in  $\text{Hyb}_{9+j}$  and by  $\mathcal{A}_{\text{NIZK}}$  can be split into parts depending only on  $sid'$  variables and the rest, where the part depending on  $sid'$  is identical for both  $\text{Hyb}_{9+j}$  and  $\mathcal{A}_{\text{NIZK}}$ . Since observability in this case depends solely on checking  $sid'$  variables, the behaviour of  $\mathcal{A}_{\text{NIZK}}$  and  $\text{Hyb}_{9+j}$  is identical.

Let  $\hat{X}$  be the set of variables defined by stand-alone symbolic Shoup's GGM functionality  $\mathbf{s}\mathcal{G}$  on group operation queries made either by  $(\text{Setup}, P)$  algorithms or  $sid$  queries forwarded by  $\mathcal{A}_{\text{NIZK}}$ . Let  $\hat{Y}$  be the set of variables defined on group operation queries forwarded by  $\mathcal{A}_{\text{NIZK}}$  for session  $sid'$ . Clearly,  $X_{sid'} \leftrightarrow \hat{Y}$  (there is a bijection between these two variable sets). Moreover,  $X_{sid} \subseteq \hat{X}$  and each  $X \in X_\pi$  can be rewritten as a linear polynomial  $p(\hat{X})$  with zero constant term ( $\deg_{\hat{X}}(p) = 0$ ). Barring observability, this is identical to how these polynomials and variables would be defined even in the UC group functionality  $\mathbf{s}\mathcal{G}$ .

Let the polynomials corresponding to elements  $g_1, g_2$  as defined by  $\mathcal{A}_{\text{NIZK}}$  be  $p_{g_i}(X_{sid}, X_\pi, X_{sid'})$ , where both polynomials are linear. The symbolic representation of the output  $h$  is defined as  $a_1 p_{g_1} + a_2 p_{g_2}$ , which is also a linear polynomial and hence can be rewritten as  $f := f^{(1)}(X_{sid}, X_\pi) + f^{(2)}(X_{sid'})$ , where  $f^{(1)}$  only depends on variables  $X_{sid}, X_\pi$ , and  $f^{(2)}$  contains the rest. Because of the observation above,  $f^{(1)}(X_{sid}, X_\pi)$  can be equivalently written as  $p(\hat{X})$  (for some linear polynomial  $p$  with zero constant term), and  $f^{(2)}(X_{sid'})$  can simply be rewritten with variables  $\hat{Y}$  keeping the coefficients unchanged. The representation of the output  $h$  becomes:

$$p_h(\hat{X}, \hat{Y}) = p(\hat{X}) + f^{(2)}(\hat{Y})$$

Note that this is exactly how the polynomial representation for  $h$  would be defined in  $\text{Hyb}_{9+j}$ . Since  $\mathcal{A}_{\text{NIZK}}$  maintains the full symbolic representation for all previous queries w.r.t  $sid'$ , it can compute  $f^{(2)}$  explicitly and locally verify if it is a zero polynomial or not. Hence, deciding observability for  $\text{OP}_{sid}$  queries is identical in  $\text{Hyb}_{9+j}$  and in the view generated by  $\mathcal{A}_{\text{NIZK}}$ .

**Linear Operations in  $sid'$ :**  $h \leftarrow \text{OP}_{sid'}(i, (g_1, a_1), (g_2, a_2))$ .

Following the argument for the previous case, polynomial representation for  $h$  as defined in  $\text{Hyb}_{9+j}$  would be  $p_h(\hat{X}, \hat{Y}) = p(\hat{X}) + f^{(2)}(\hat{Y})$  while for  $\mathcal{A}_{\text{NIZK}}$  it is  $f^{(1)}(X_{sid}, X_\pi) + f^{(2)}(X_{sid'})$ . We have  $f^{(1)}(X_{sid}, X_\pi) = p(\hat{X})$ , and that the map  $\tau$  is defined identically by  $\mathcal{A}_{\text{NIZK}}$  and  $\mathbf{s}\mathcal{G}$  (and hence by the group functionality in  $\text{Hyb}_{9+j}$ ). Thus,  $f^{(1)}(\tau(X_{sid}), \tau(X_\pi)) = \tau(0)$  iff  $p(\hat{X})$  is a zero polynomial. The LHS is the check performed by  $\mathcal{A}_{\text{NIZK}}$ , while the RHS is the check performed in  $\text{Hyb}_{9+j}$ . Hence, deciding observability is again identical for  $\mathcal{A}_{\text{NIZK}}$  and  $\text{Hyb}_{9+j}$ .

**Pairing Operations in  $sid$ :**  $h \leftarrow \text{PAIR}_{sid}(g_1, g_2)$ .

The polynomial representation of  $h$  as defined by  $\mathcal{A}_{\text{NIZK}}$  can be written as  $f := f^{(1)}(X_{sid}, X_\pi) + f^{(2)}(X_{sid}, X_\pi, X_{sid'})$ . The term  $f^{(2)}$  can be further split as  $f^{(3)}(X_{sid'}) \cdot (f^{(4)}(X_{sid}) + f^{(5)}(X_\pi)) + f^{(6)}(X_{sid'})$  (coefficients for  $f$  polynomials here are fully determined by  $\mathcal{A}_{\text{NIZK}}$ ).

Similar to case of linear operations, variables  $X_{sid} \cup X_\pi$  can be replaced by appropriate variables in  $\hat{X}$  and variables  $X_{sid'}$  by  $\hat{Y}$ , i.e., there exists  $p_1, \dots, p_4$  s.t.

$$p_h(\hat{X}, \hat{Y}) := p_1(\hat{X}) + p_2(\hat{Y}) \cdot p_3(\hat{X}) + p_4(\hat{Y})$$

where after substituting the variables appropriately,  $p_1$  is equivalent to  $f^{(1)}$ ;  $p_2$  to  $f^{(3)}$ ,  $p_3$  to  $f^{(4)} + f^{(5)}$ , and  $p_4$  is equivalent to the term  $f^{(6)}$ .

To decide observability two checks are performed:

1.  $\mathcal{A}_{\text{NIZK}}$  locally checks if the polynomial  $f^{(6)}(X_{sid'})$  is a zero polynomial. Since  $\mathcal{A}_{\text{NIZK}}$  maintains full representation for  $sid'$  variables it can do this. On the other hand,  $\text{Hyb}_{9+j}$  checks if  $p_4(\hat{Y})$  is zero. Since  $p_4$  and  $f^{(6)}$  are equivalent, these result in identical checks.
2.  $\mathcal{A}_{\text{NIZK}}$  checks  $f^{(3)}$  is non zero polynomial locally or  $(f^{(4)}(\tau(X_{sid})) + f^{(5)}(\tau(X_\pi))) = \tau(0)$  (by querying  $\mathbf{s}\text{-G}$  for computing group operations). Again, since  $\tau$ 's are defined identically (as in the linear case), and because of the equivalence between polynomials  $p$  defined by  $\text{Hyb}_{9+j}$  and  $f$  defined by  $\mathcal{A}_{\text{NIZK}}$ , this results in identical outputs.

**Pairing Operations in  $sid'$ :**  $h \leftarrow \text{PAIR}_{sid'}(g_1, g_2)$  As in the previous case, here too the behaviour of  $\mathcal{A}_{\text{NIZK}}$  will be identical to that of  $\text{Hyb}_{9+j}$ .

**Claim 2.** For  $j \in \{0, \dots, s_2 - 1\}$   $\text{Hyb}_{9+s_1+j}$  and  $\text{Hyb}_{9+s_1+j+1}$  are indistinguishable.

*Proof.* The difference between the hybrids is that the extractor may fail to output a valid witness in  $\text{Hyb}_{9+s_1+j+1}$  while the verifier accepts. This we reduce to simulation-extractability of  $\Pi$ . Given  $\mathcal{Z}$  who distinguishes between the two hybrids, we build an adversary  $\mathcal{A}_{\text{SE}}$  whose code is the same as  $\mathcal{A}^*$ . The argument that  $\mathcal{A}$  generates an indistinguishable view is the same as already made in the NIZK case.

If a proof verifies in the UC experiment, it will also verify in the SIMEXT experiment (since all oracle queries are forwarded). All we need to argue is that the view of the extractors  $(E_0, E_1)$  in the SIMEXT experiment is identical to that of the extractors in the UC experiment in hybrid  $\text{Hyb}_{9+s_1+j+1}$ . Hence, if it fails to extract a valid witness in the latter, then this fails even in the SIMEXT, breaking the assumption. The difference in the view of the extractors in the two experiments is that in SIMEXT game, the extractors learn the entire group oracle transcript (including  $sid'$  queries), while in the hybrid they only learn  $Ob$  which is a strict subset. Hence, we must rule out the possibility of valid extraction given this extra information.

The inputs to  $E_0$  is identical in both experiments and so the output  $(i, h_j, B_j)$  outputs are identical. We claim that the outputs of  $\text{FINDREP}(\cdot, Ob_{sid})$  and  $\text{FINDREP}(\cdot, \text{tr}_G)$  are identical. Given this, the output of  $E_1$  in both experiments will be identical.

Recall how the algorithm  $\text{FINDREP}(1/2, \cdot)$  works: it parses  $B := (B_1, \dots, B_n) \in S_i^n$ . Then it initializes a vector  $Rep$  as  $Rep[h] \leftarrow 0^n, \forall h \in S_i$ , and for  $j \in [n]$ ,  $Rep[B_j] = (0, \dots, 1, 0, \dots, 0)$  where  $Rep[B_j]$  is 1 at position  $j$  and 0 otherwise. These steps remain identical for both experiments. The next step is to execute  $\forall ob = (\text{OP}, i, g_1, g_2, a_1, a_2, h) \in \text{tr}$ , update  $Rep[h] := a_1 Rep[g_1] + a_2 Rep[g_2]$ , where  $\text{tr}$  is the query tuples taken as input. A difference may now occur if  $Rep[h]$  is set differently given access to a larger  $\text{tr}$ .

Let  $\text{tr}_G$  consist of all group queries ever made in the SIMEXT experiment. This includes all queries by the adversary  $\mathcal{A}_{\text{SE}}$  and the ones internal algorithms of the SIMEXT challenger (by  $\text{Setup}, P, V$  interfaces). We can partition  $\text{tr}_G$  as  $\text{tr}_{ob} || \text{tr}_{se} || \text{tr}_{\bar{ob}}$ , where  $\text{tr}_{ob}$  denotes queries in  $Ob$  (as maintained by  $\mathcal{A}_{\text{SE}}$ ),  $\text{tr}_{se}$  are queries by SIMEXT interfaces, and  $\text{tr}_{\bar{ob}}$  the rest (this only includes legal queries for  $sid'$  session maintained by  $\mathcal{A}_{\text{SE}}$ ). Notice that this partitioning is perfect as the symbolic representation of a group element is guaranteed to be unique and that in the hybrid experiment, query tuples given as input to  $\text{FINDREP}$  will be equal to  $\text{tr}_{ob} || \text{tr}_{se}$ , while in the reduction it will be  $\text{tr}_{ob} || \text{tr}_{se} || \text{tr}_{\bar{ob}}$ . We will proceed by contradiction to show that  $\forall (\text{OP}, i, g_1, g_2, a_1, a_2, h) \in \text{tr}_{\bar{ob}}, Rep[h] = 0$ . If there exists  $h$  such that  $Rep[h] \neq 0$  then it means that  $Rep[g_1] \neq 0$  and/or  $Rep[g_2] \neq 0$ , meaning that  $g_1$  and/or  $g_2$  must be computed via bases elements. Hence, contradicting the fact that  $h$  is not observable.

For pairing operations we will follow the same blueprint, parse  $B^{(i)}$  as  $(B_1^{(i)}, \dots, B_{n_i}^{(i)}) \in S_i^{n_i}$  for all  $i \in 1, 2, t$  and initialize the corresponding  $Rep_1, Rep_2$  as previously. Initialize  $Rep[h] \leftarrow 0^n, \forall h \in S_i$  and for  $j \in [n_t + n_1 \cdot n_2], Rep[B_j^{(i)}][j] = (0, \dots, 1, 0, \dots, 0)$  where  $Rep[B_j^{(i)}]$  is 1 at position  $j$  and 0 otherwise.  $Rep[h]$  is updated as previously for operations using the group law and for pairing operations we will have  $Rep[h] := Rep_t(0^{n_t}, Rep_1[g_1] \otimes Rep_2[g_2])$ . Then we partition the entire transcript as before to obtain:  $tr_{ob} || tr_{se} || tr_{ob}$ . As  $\mathbb{Z}_p$  has no zero divisors, the only way for the tensor product  $Rep[g_1] \otimes Rep[g_2]$  to be non-zero is to have  $Rep[g_1] \neq 0$  and  $Rep[g_2] \neq 0$ , therefore for all pairing operation  $(PAIR, g_1, g_2, h) \in tr_{ob}, Rep[h] = \mathbf{0}$  otherwise they would be observable.

## 6 Inferring GGM security from AGM security

Most security analysis are in FKL-AGM, i.e. in the AGM model defined by Fuchsbauer, Kiltz, and Loss in [34]. Although this model is meant to be close to Shoup's GGM one cannot always conclude security in the GGM given security in the AGM as shown in [62, 64]. However, [45] showed that this link holds between AGM and Maurer's GGM and to formalise it they introduce a pseudocode computation framework. In this section, we extend this framework to include accesses to random oracle, hence making it applicable to zkSNARKs. Then, we complete the picture by lifting the security in this variant of GGM to Shoup's GGM.

### 6.1 Type Safe Oracle Framework

**Data-types.** Data types restrict how an algorithm can interact with specific variables. We consider two data types  $\langle \text{bit} \rangle$  and  $\langle \text{elem} \rangle$  that can be seen as containing the discrete logarithm in the group.  $\langle \text{bits} \rangle$  represents a sequence of bits of arbitrary size.

**Typed Random Oracle.** A typed random oracle defined in Definition 3 is a natural generalization of the classical ROM. In this model, the oracle can be queried with group elements directly rather than first having to convert them to their representation in bits. This means that a query can be deterministically parsed as a series of group elements and bit strings. While this may seem arbitrary, this assumption already appears implicitly in prior works [9, 18, 35], where the ROM is defined as  $\mathbb{G} \times \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$  (with minor modifications).

**Definition 3 (Typed Random Oracle).** A typed random oracle is  $RO_{ts} : ((\langle \text{elem} \rangle e, \langle \text{bits} \rangle s)^*) \rightarrow \langle \text{bits} \rangle r$  that takes a list of tuple  $e \in \mathbb{G} \cup \{\emptyset\}$  and a bit string  $s \in \{0, 1\}^* \cup \{\emptyset\}$  and outputs a uniformly sampled bit string  $r$  of size  $n$ .

It is easy to see that given a typed-random oracle we can generalize the pseudocode framework to algorithms that require access to a random oracle. Intuitively, this does not break the “type-safeness” because the random-oracle itself is typed, and can be directly queried on group elements rather than requiring the algorithm to interpret group elements as bit strings (which would be the case in classical random oracles). Notice that this means that a typed random oracle queried on the bitstring encoding or on an element would yield different output which is normal as they should be seen as two different inputs.

In many cases, typed-random oracles can also easily replace the usual random oracles without affecting the security, or requiring modifications to the algorithms in the idealised settings (as we will see later in the case studies for Plonk and Marlin). A real-world example where this is not the case is ECDSA [58], whose signer algorithm necessarily requires concrete representation of the group elements.

**Universe of Computation [45].** Let  $S$  be a set of elements,  $\perp$  be a special element used to indicate rejection, and  $S_\perp = S \cup \{\perp\}$ . Let  $\Pi$  be a set of functions of the form  $S_\perp \times S_\perp \times \dots \times S_\perp \times \{0, 1\}^n \rightarrow S_\perp$ ,

**Figure 3:** Types of Oracles Accesses

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Oracle</b> $\mathcal{O}_f(\langle \text{elem} \rangle l_1, \langle \text{elem} \rangle l_2, \langle \text{bits} \rangle a, \langle \text{bits} \rangle b)$<br>1: <b>assert</b> $a, b \in \mathbb{Z}_p^2$<br>2: Compute $l \leftarrow a \cdot l_1 + b \cdot l_2$<br>3: <b>return</b> $\langle \text{elem} \rangle l$<br><b>Oracle</b> $\mathcal{O}_\rho(\langle \text{elem} \rangle l_1, \langle \text{elem} \rangle l_2)$<br>1: If $l_1, l_2$ contain the same $x \in \mathbb{Z}_p$ , <b>return</b> 1.<br>2: Otherwise, <b>return</b> 0. | <b>Oracle</b> $\mathcal{O}_\sigma(\langle \text{elem} \rangle l)$<br>1: <b>return</b> $\langle \text{bit} \rangle a \leftarrow \sigma(l)$<br><b>Oracle</b> $\mathcal{O}_{\sigma^{-1}}(\langle \text{bit} \rangle b_1, \dots, \langle \text{bit} \rangle b_n)$<br>1: <b>return</b> $\langle \text{elem} \rangle g = \sigma^{-1}(\mathbf{b})$ .<br><b>Oracle</b> $\text{RO}_{\text{ts}}(x)$<br>1: $T$ is initially empty table<br>2: Parse $x$ as series of $\langle \text{elem} \rangle, \langle \text{bits} \rangle$ variables<br>3: <b>if</b> $x \in T$ : <b>then</b><br>4: <b>return</b> $T[x]$<br>5: <b>end if</b><br>6: Sample $y \xleftarrow{\$} \{0, 1\}^\lambda$<br>7: Store $T[x] \leftarrow y$<br>8: <b>return</b> $y$ |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

$\Sigma$  be a set of relations of the form  $S_\perp \times S_\perp \times \dots \times S_\perp \times \{0, 1\}^n \rightarrow \{0, 1\}$ , and an injection  $\sigma : S_\perp \rightarrow \{0, 1\}^*$ , and  $\sigma^{-1}$  be the corresponding inverse mapping. We will consider that there exists an encoding for the bottom element  $\perp$ . And let  $\text{RO}_{\text{ts}}$  be a typed random oracle as defined in Definition 3.

A universe of computation  $U_\sigma$  specifies the tuple  $(S, \Pi, \Sigma, \sigma)$ . We write  $U$  (dropping  $\sigma$ ) when  $\sigma$  can be left undefined as the algorithms are agnostic of  $\sigma$ . Note that we do not include  $\text{RO}_{\text{ts}}$  in this tuple. This is because  $U_\sigma$  could be shared by several algorithms (as modelled in the UC setting), while  $\text{RO}_{\text{ts}}$  could be local to an algorithm.

Let  $\mathcal{O}_f$  be an oracle for each  $f \in \Pi$ , and  $\mathcal{O}_\rho$  for each  $\rho \in \Sigma$  in Figure 3. Algorithms can perform two types of function evaluations: (1) functions that operate over bits i.e., functions  $f_b$  that map  $\langle \text{bit} \rangle$  data-types to  $\langle \text{bit} \rangle$ . These functions cannot take  $\langle \text{elem} \rangle$  data-types as inputs and can be computed locally; (2) special functions (listed in Figure 3) whose input and/or output data-types include  $\langle \text{elem} \rangle$  types. To compute these, algorithms must make oracle to Figure 3. Finally, algorithms have access to a typed random oracle  $\text{RO}_{\text{ts}}$  also listed in Figure 3.

A syntactic difference from [45] is that we directly define our framework in an oracle framework, where algorithms make explicit oracle calls to perform certain computation rather than “using” a certain function locally as in the pseudocode framework<sup>8</sup>. Our direct approach retains the ease of checking type-safeness of algorithms as in the pseudocode framework, but allows for observing oracle queries which will be required in the UC setting later. Moreover, it allows a simpler incorporation of random oracles in the framework.

**Hierarchy of algorithms.** The usual a hierarchy of algorithms [45] without considering  $\text{RO}_{\text{ts}}$  can be defined as follows. At the lowest level is type-safe GGM ( $\text{TSA}[U]$ ) where (type-safe) algorithms only have access to  $(\mathcal{O}_f, \mathcal{O}_\rho)$ . This is followed by type-safe AGM ( $\text{AA}[U_\sigma]$ ) where (algebraic) algorithms have additional access to  $\mathcal{O}_\sigma$  and hence can obtain group element representation, but are still restricted to using  $\mathcal{O}_f$  to do any sort of group operation. Finally, at the top of the hierarchy are standard algorithms ( $\text{SA}[U_\sigma]$ ) that also have access to  $\mathcal{O}_{\sigma^{-1}}$  allowing them to perform local group operations. Thus, we have  $\text{TSA}[U] \subset \text{AA}[U_\sigma] \subset \text{SA}[U_\sigma]$  for an *efficient* encoding  $\sigma$ .

As stated earlier it might seem counter-intuitive at first to give a random oracle access to a type-safe model, as algorithms can use the typed random oracle to obtain an encoding for a group

<sup>8</sup> Our oracle framework is different than the oracle framework defined in [45], which is meant to be a faithful formulation of Maurer’s GGM. As a matter of fact it is *stateful* which can be problematic as observed by Zhandry et al. in [62]. Our oracle framework is much closer to the pseudocode framework of [45]. The only difference is that instead of viewing them as functions, we view them as oracles.

element. We observe that  $\text{TSA}[U]^{\text{RO}_{\text{ts}}} \cong \text{AA}[U_{\bar{\sigma}}]$ , i.e. giving access to a typed random oracle to a type safe algorithm is equivalent to giving access to a randomized encoding  $\bar{\sigma}$  to an algebraic algorithm. Furthermore, for an algorithm in  $\text{AA}[U_{\bar{\sigma}}]$ , any group operation must still be performed via queries to  $\mathcal{O}_f, \mathcal{O}_\rho$  (since it does not have access to  $\bar{\sigma}^{-1}$ ), i.e.,  $\text{TSA}[U] \subset \text{AA}[U_{\bar{\sigma}}]$ . We also have that  $\text{TSA}[U] \subset \text{TSA}[U]^{\text{RO}_{\text{ts}}}$  and  $\text{AA}[U_{\bar{\sigma}}] \subset \text{AA}^{\text{RO}_{\text{ts}}}[U_\sigma]$  (former has access to a strict subset of oracles compared to the latter). Hence, we get  $\text{TSA}[U] \subset \text{TSA}[U]^{\text{RO}_{\text{ts}}} \subset \text{AA}^{\text{RO}_{\text{ts}}}[U_\sigma]$ . From this observation we can derive the augmented hierarchy of [Definition 4](#).

We state the analogous lifting theorem in [Theorem 5](#) (relevant preliminaries can be found in [Appendix F](#)). Informally, it states that assume that a primitive  $\Pi$  is shown to be secure against *algebraic* adversaries via a reduction  $R$  to an underlying problem. If the reduction is a type-safe algorithm whenever the adversary is type-safe, and if the underlying problem is hard against type-safe adversaries, then  $\Pi$  is secure against *type-safe* adversaries.

**Definition 4.** (*Hierarchy of Algorithms*) Fix universe  $U$ , and let  $\text{RO}_{\text{ts}}$  be a typed-random oracle. Let [Figure 3](#) specify functions w.r.t  $U$ . Then,

- $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  or *Type-safe algorithms (w.r.t  $\text{RO}_{\text{ts}}$ )* is the set of all oracle algorithms that have access to oracles  $\mathcal{O}_f, \mathcal{O}_\rho$  and  $\mathcal{O}_{\sigma_{\text{RO}}}$  only.
- $\text{AA}^{\text{RO}_{\text{ts}}}[U_\sigma]$  or *Algebraic algorithms (w.r.t  $\text{RO}_{\text{ts}}$ )* is the set of all oracle algorithms that have access to oracle  $\mathcal{O}_f, \mathcal{O}_\rho, \mathcal{O}_{\sigma_{\text{RO}}}$ , and  $\mathcal{O}_\sigma$ .
- $\text{SA}^{\text{RO}_{\text{ts}}}[U_\sigma]$  or *Standard algorithms* is the set of all oracle algorithms that have access to all the oracles in [Figure 3](#).

**Theorem 5.** Fix universe  $U$  and advantage metrics  $\text{Adv}_U^{\text{met}_1}, \text{Adv}_U^{\text{met}_2}$ , where  $\text{Adv}_U^{\text{met}_2}$  is generically hard. Let  $R$  be  $\Delta$ -AGM reduction for this metric that is model preserving and efficiency preserving. Then  $\text{Adv}_U^{\text{met}_1}$  is generically hard.

**Lifting to security in Shoup’s GGM.** We observe that a similar hierarchy (as for [Definition 4](#)) holds if one replaces the efficient encoding by a random encoding  $\hat{\sigma}$  (as needed in shoup’s GGM) and therefore we obtain a hierarchy all the way up to Shoup’s GGM where  $\text{TSA}[U]^{\text{RO}_{\text{ts}}}$  is the link.

**Theorem 6.** If  $Ch$  is a single-stage game and  $\Pi$  satisfies  $Ch$  property in the  $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  model. Then,  $\Pi'$  satisfies  $Ch'$  property in  $\text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$ , where  $Ch'$  and  $\Pi'$  are canonical translations of  $Ch, \Pi$  with respect to a randomly chosen encoding  $\hat{\sigma}$ .

In [Theorem 6](#),  $\text{RR}^{\text{RO}}$  denotes the set of algorithms with access to Shoup’s GGM and standard ROM. A canonical transformation, informally converts an algorithm in  $\text{TSA}^{\text{RO}_{\text{ts}}}$  to a related algorithm in  $\text{RR}^{\text{RO}}$ .

Moreover, as in [\[62\]](#) we also show equivalence (for single-stage games) between TSA and RR models by showing,

**Theorem 7.** If  $Ch'$  is secure in  $\text{RR}^{\text{RO}}[U]_{\hat{\sigma}}$ , then  $Ch$  is secure in the  $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  model.

We detail in [Appendix G](#) the proofs of [Theorem 5](#), [Theorem 6](#) and [Theorem 7](#).

## 7 Implications and Open Questions

**Implications of our framework.** We sketch in [Appendix H](#) how we conclude UC security for two important proof systems – Plonk [\[36\]](#) and Marlin [\[27\]](#).



The result follows from typing the SIMEXT result established by [33] in the AGM + non-programmable ROM where they show how to infer the aforementioned property for zkSNARKS obtained from a PIOP compiled using KZG [46]. By analyzing three core theorems from this paper, we identify a sequence of type-safe reductions to the underlying generically hard problem. Then applying type-safe AGM to type-safe GGM lifting theorem (Theorem 5), type-safe GGM to shoup’s GGM implication (Theorem 6), and finally the UC lifting theorem (Theorem 4), we conclude UC security for Plonk and Marlin.

**Future Work.** We find a few interesting directions for extensions. First, a natural question is to extend the formalization in this paper to capture security against adaptive adversaries and, since we are in the GGM, achieve unconditional security (as is done in [22, 26]). Second, we believe our framework can be applied to the class of transparent, discrete-log based proof systems like [15, 60] in order to show their UC security. Finally, a more interesting, and practically relevant question is to cover the scenario where group-based zkSNARKs can safely reuse parts of the CRS. This question is motivated by practical scenarios where a KZG-based zkSNARK might use a powers-of-tau CRS that is also used by other zkSNARKs or cryptographic primitives (e.g., vector commitments). This scenario escapes our current modeling since it would need to work with a global CRS functionality where, however, the elements cannot belong to any particular session. This extension might require redefining observability for the global-GGM functionality as well as identifying conditions in the zkSNARK design under which such a reuse of the CRS is safe.

## References

1. Abdalla, M., Barbosa, M., Katz, J., Loss, J., Xu, J.: Algebraic adversaries in the universal composability framework. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021, Part III*. Lecture Notes in Computer Science, vol. 13092, pp. 311–341. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). [https://doi.org/10.1007/978-3-030-92078-4\\_11](https://doi.org/10.1007/978-3-030-92078-4_11)
2. Abdolmaleki, B., Glaeser, N., Ramacher, S., Slamanig, D.: Circuit-succinct universally-composable NIZKs with updatable CRS. In: *CSF 2024: IEEE 37th Computer Security Foundations Symposium*. pp. 527–542. IEEE Computer Society Press, Enschede, The Netherlands (Jul 8–12, 2024). <https://doi.org/10.1109/CSF61375.2024.00006>
3. Abdolmaleki, B., Ramacher, S., Slamanig, D.: Lift-and-shift: Obtaining simulation extractable subversion and updatable SNARKs generically. In: Ligatti, J., Ou, X., Katz, J., Vigna, G. (eds.) *ACM CCS 2020: 27th Conference on Computer and Communications Security*. pp. 1987–2005. ACM Press, Virtual Event, USA (Nov 9–13, 2020). <https://doi.org/10.1145/3372297.3417228>
4. Aranha, D.F., Bennedsen, E.M., Campanelli, M., Ganesh, C., Orlandi, C., Takahashi, A.: ECLIPSE: Enhanced compiling method for Pedersen-committed zkSNARK engines. In: Hanaoka, G., Shikata, J., Watanabe, Y. (eds.) *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part I*. Lecture Notes in Computer Science, vol. 13177, pp. 584–614. Springer, Cham, Switzerland, Virtual Event (Mar 8–11, 2022). [https://doi.org/10.1007/978-3-030-97121-2\\_21](https://doi.org/10.1007/978-3-030-97121-2_21)
5. Badertscher, C., Campanelli, M., Ciampi, M., Russo, L., Siniscalchi, L.: Universally composable SNARKs with transparent setup without programmable random oracle. In: Kalai, Y.T., Kamara, S.F. (eds.) *Advances in Cryptology – CRYPTO 2025, Part VII*. Lecture Notes in Computer Science, vol. 16006, pp. 225–258. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2025). [https://doi.org/10.1007/978-3-032-01907-3\\_8](https://doi.org/10.1007/978-3-032-01907-3_8)
6. Badertscher, C., Canetti, R., Hesse, J., Tackmann, B., Zikas, V.: Universal composition with global subroutines: Capturing global setup within plain UC. In: Pass, R., Pietrzak, K. (eds.) *TCC 2020: 18th Theory of Cryptography Conference, Part III*. Lecture Notes in Computer Science, vol. 12552, pp. 1–30. Springer, Cham, Switzerland, Durham, NC, USA (Nov 16–19, 2020). [https://doi.org/10.1007/978-3-030-64381-2\\_1](https://doi.org/10.1007/978-3-030-64381-2_1)
7. Baghery, K., Kohlweiss, M., Siim, J., Volkhov, M.: Another look at extraction and randomization of groth’s zk-SNARK. In: Borisov, N., Díaz, C. (eds.) *FC 2021: 25th International Conference on Financial Cryptography and Data Security, Part I*. Lecture Notes in Computer Science, vol. 12674, pp. 457–475. Springer Berlin Heidelberg, Germany, Virtual Event (Mar 1–5, 2021). [https://doi.org/10.1007/978-3-662-64322-8\\_22](https://doi.org/10.1007/978-3-662-64322-8_22)
8. Baghery, K., Sedaghat, M.: Tiramisu: Black-box simulation extractable NIZKs in the updatable CRS model. In: Conti, M., Stevens, M., Krenn, S. (eds.) *CANS 21: 20th International Conference on Cryptology and Network*

- Security. Lecture Notes in Computer Science, vol. 13099, pp. 531–551. Springer, Cham, Switzerland, Vienna, Austria (Dec 13–15, 2021). [https://doi.org/10.1007/978-3-030-92548-2\\_28](https://doi.org/10.1007/978-3-030-92548-2_28)
9. Barkan-Vered, W., Harding, F., Keller, J., Xu, J.: On the non-malleability of ECVRF in the algebraic group model. Cryptology ePrint Archive, Report 2023/1004 (2023), <https://eprint.iacr.org/2023/1004>
  10. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable zero knowledge with no trusted setup. In: Boldyreva, A., Micciancio, D. (eds.) *Advances in Cryptology – CRYPTO 2019, Part III*. Lecture Notes in Computer Science, vol. 11694, pp. 701–732. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2019). [https://doi.org/10.1007/978-3-030-26954-8\\_23](https://doi.org/10.1007/978-3-030-26954-8_23)
  11. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent succinct arguments for R1CS. In: Ishai, Y., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2019, Part I*. Lecture Notes in Computer Science, vol. 11476, pp. 103–128. Springer, Cham, Switzerland, Darmstadt, Germany (May 19–23, 2019). [https://doi.org/10.1007/978-3-030-17653-2\\_4](https://doi.org/10.1007/978-3-030-17653-2_4)
  12. Ben-Sasson, E., Chiesa, A., Spooner, N.: Interactive oracle proofs. In: Hirt, M., Smith, A.D. (eds.) *TCC 2016-B: 14th Theory of Cryptography Conference, Part II*. Lecture Notes in Computer Science, vol. 9986, pp. 31–60. Springer Berlin Heidelberg, Germany, Beijing, China (Oct 31 – Nov 3, 2016). [https://doi.org/10.1007/978-3-662-53644-5\\_2](https://doi.org/10.1007/978-3-662-53644-5_2)
  13. Bobolz, J., Farshim, P., Kohlweiss, M., Takahashi, A.: The brave new world of global generic groups and UC-secure zero-overhead SNARKs. In: Boyle, E., Mahmoody, M. (eds.) *TCC 2024: 22nd Theory of Cryptography Conference, Part I*. Lecture Notes in Computer Science, vol. 15364, pp. 90–124. Springer, Cham, Switzerland, Milan, Italy (Dec 2–6, 2024). [https://doi.org/10.1007/978-3-031-78011-0\\_4](https://doi.org/10.1007/978-3-031-78011-0_4)
  14. Bootle, J., Cerulli, A., Chaidos, P., Groth, J., Petit, C.: Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In: Fischlin, M., Coron, J.S. (eds.) *Advances in Cryptology – EUROCRYPT 2016, Part II*. Lecture Notes in Computer Science, vol. 9666, pp. 327–357. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). [https://doi.org/10.1007/978-3-662-49896-5\\_12](https://doi.org/10.1007/978-3-662-49896-5_12)
  15. Bünz, B., Bootle, J., Boneh, D., Poelstra, A., Wuille, P., Maxwell, G.: Bulletproofs: Short proofs for confidential transactions and more. In: *2018 IEEE Symposium on Security and Privacy*. pp. 315–334. IEEE Computer Society Press, San Francisco, CA, USA (May 21–23, 2018). <https://doi.org/10.1109/SP.2018.00020>
  16. Camenisch, J., Drijvers, M., Gagliardoni, T., Lehmann, A., Neven, G.: The wonderful world of global random oracles. In: Nielsen, J.B., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2018, Part I*. Lecture Notes in Computer Science, vol. 10820, pp. 280–312. Springer, Cham, Switzerland, Tel Aviv, Israel (Apr 29 – May 3, 2018). [https://doi.org/10.1007/978-3-319-78381-9\\_11](https://doi.org/10.1007/978-3-319-78381-9_11)
  17. Campanelli, M., Faonio, A., Fiore, D., Querol, A., Rodríguez, H.: Lunar: A toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021, Part III*. Lecture Notes in Computer Science, vol. 13092, pp. 3–33. Springer, Cham, Switzerland, Singapore (Dec 6–10, 2021). [https://doi.org/10.1007/978-3-030-92078-4\\_1](https://doi.org/10.1007/978-3-030-92078-4_1)
  18. Campanelli, M., Fiore, D., Pancholi, M.: When can we incrementally prove computations of arbitrary depth? Cryptology ePrint Archive, Report 2025/1413 (2025), <https://eprint.iacr.org/2025/1413>
  19. Campanelli, M., Fiore, D., Querol, A.: LegoSNARK: Modular design and composition of succinct zero-knowledge proofs. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *ACM CCS 2019: 26th Conference on Computer and Communications Security*. pp. 2075–2092. ACM Press, London, UK (Nov 11–15, 2019). <https://doi.org/10.1145/3319535.3339820>
  20. Campanelli, M., Ganesh, C., Khoshakhlagh, H., Siim, J.: Impossibilities in succinct arguments: Black-box extraction and more. In: El Mrabet, N., De Feo, L., Duquesne, S. (eds.) *AFRICACRYPT 23: 14th International Conference on Cryptology in Africa*. Lecture Notes in Computer Science, vol. 14064, pp. 465–489. Springer, Cham, Switzerland, Sousse, Tunisia (Jul 19–21, 2023). [https://doi.org/10.1007/978-3-031-37679-5\\_20](https://doi.org/10.1007/978-3-031-37679-5_20)
  21. Canetti, R.: Universally composable security: A new paradigm for cryptographic protocols. In: *42nd Annual Symposium on Foundations of Computer Science*. pp. 136–145. IEEE Computer Society Press, Las Vegas, NV, USA (Oct 14–17, 2001). <https://doi.org/10.1109/SFCS.2001.959888>
  22. Canetti, R., Chen, M.: Universally composable succinct vector commitments and applications. In: Applebaum, B., Lin, H.R. (eds.) *TCC 2025: 23rd Theory of Cryptography Conference, Part IV*. Lecture Notes in Computer Science, vol. 16271, pp. 642–674. Springer, Cham, Switzerland, Aarhus, Denmark (Dec 1–5, 2025). [https://doi.org/10.1007/978-3-032-12290-2\\_21](https://doi.org/10.1007/978-3-032-12290-2_21)
  23. Canetti, R., Jain, A., Scafuro, A.: Practical UC security with a global random oracle. In: Ahn, G.J., Yung, M., Li, N. (eds.) *ACM CCS 2014: 21st Conference on Computer and Communications Security*. pp. 597–608. ACM Press, Scottsdale, AZ, USA (Nov 3–7, 2014). <https://doi.org/10.1145/2660267.2660374>
  24. Canetti, R., Lindell, Y., Ostrovsky, R., Sahai, A.: Universally composable two-party and multi-party secure computation. In: *34th Annual ACM Symposium on Theory of Computing*. pp. 494–503. ACM Press, Montréal, Québec, Canada (May 19–21, 2002). <https://doi.org/10.1145/509907.509980>

25. Canetti, R., Sarkar, P., Wang, X.: Triply adaptive UC NIZK. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part II. Lecture Notes in Computer Science*, vol. 13792, pp. 466–495. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). [https://doi.org/10.1007/978-3-031-22966-4\\_16](https://doi.org/10.1007/978-3-031-22966-4_16)
26. Chiesa, A., Fenzi, G.: zkSNARKs in the ROM with unconditional UC-security. In: Boyle, E., Mahmoody, M. (eds.) *TCC 2024: 22nd Theory of Cryptography Conference, Part I. Lecture Notes in Computer Science*, vol. 15364, pp. 67–89. Springer, Cham, Switzerland, Milan, Italy (Dec 2–6, 2024). [https://doi.org/10.1007/978-3-031-78011-0\\_3](https://doi.org/10.1007/978-3-031-78011-0_3)
27. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, P., Ward, N.P.: Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In: Canteaut, A., Ishai, Y. (eds.) *Advances in Cryptology – EUROCRYPT 2020, Part I. Lecture Notes in Computer Science*, vol. 12105, pp. 738–768. Springer, Cham, Switzerland, Zagreb, Croatia (May 10–14, 2020). [https://doi.org/10.1007/978-3-030-45721-1\\_26](https://doi.org/10.1007/978-3-030-45721-1_26)
28. De Santis, A., Di Crescenzo, G., Ostrovsky, R., Persiano, G., Sahai, A.: Robust non-interactive zero knowledge. In: Kilian, J. (ed.) *Advances in Cryptology – CRYPTO 2001. Lecture Notes in Computer Science*, vol. 2139, pp. 566–598. Springer Berlin Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2001). [https://doi.org/10.1007/3-540-44647-8\\_33](https://doi.org/10.1007/3-540-44647-8_33)
29. Dellepère, M., Mishra, P., Shirzad, A.: Garuda and pari: Faster and smaller SNARKs via equi-efficient polynomial commitments. *Cryptology ePrint Archive, Report 2024/1245* (2024), <https://eprint.iacr.org/2024/1245>
30. Dent, A.W.: Adapting the weaknesses of the random oracle model to the generic group model. In: Zheng, Y. (ed.) *Advances in Cryptology – ASIACRYPT 2002. Lecture Notes in Computer Science*, vol. 2501, pp. 100–109. Springer Berlin Heidelberg, Germany, Queenstown, New Zealand (Dec 1–5, 2002). [https://doi.org/10.1007/3-540-36178-2\\_6](https://doi.org/10.1007/3-540-36178-2_6)
31. Elias, P.: Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory* **21**(2), 194–203 (1975). <https://doi.org/10.1109/TIT.1975.1055349>
32. Faonio, A., Fiore, D., Kohlweiss, M., Russo, L., Zajac, M.: From polynomial IOP and commitments to non-malleable zkSNARKs. In: Rothblum, G.N., Wee, H. (eds.) *TCC 2023: 21st Theory of Cryptography Conference, Part III. Lecture Notes in Computer Science*, vol. 14371, pp. 455–485. Springer, Cham, Switzerland, Taipei, Taiwan (Nov 29 – Dec 2, 2023). [https://doi.org/10.1007/978-3-031-48621-0\\_16](https://doi.org/10.1007/978-3-031-48621-0_16)
33. Faonio, A., Fiore, D., Russo, L.: Real-world universal zkSNARKs are non-malleable. In: Luo, B., Liao, X., Xu, J., Kirda, E., Lie, D. (eds.) *ACM CCS 2024: 31st Conference on Computer and Communications Security*. pp. 3138–3151. ACM Press, Salt Lake City, UT, USA (Oct 14–18, 2024). <https://doi.org/10.1145/3658644.3690351>
34. Fuchsbauer, G., Kiltz, E., Loss, J.: The algebraic group model and its applications. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology – CRYPTO 2018, Part II. Lecture Notes in Computer Science*, vol. 10992, pp. 33–62. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 19–23, 2018). [https://doi.org/10.1007/978-3-319-96881-0\\_2](https://doi.org/10.1007/978-3-319-96881-0_2)
35. Fuchsbauer, G., Plouviez, A., Seurin, Y.: Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In: Canteaut, A., Ishai, Y. (eds.) *Advances in Cryptology – EUROCRYPT 2020, Part II. Lecture Notes in Computer Science*, vol. 12106, pp. 63–95. Springer, Cham, Switzerland, Zagreb, Croatia (May 10–14, 2020). [https://doi.org/10.1007/978-3-030-45724-2\\_3](https://doi.org/10.1007/978-3-030-45724-2_3)
36. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive, Report 2019/953* (2019), <https://eprint.iacr.org/2019/953>
37. Ganesh, C., Kondi, Y., Orlandi, C., Pancholi, M., Takahashi, A., Tschudi, D.: Witness-succinct universally-composable SNARKs. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023, Part II. Lecture Notes in Computer Science*, vol. 14005, pp. 315–346. Springer, Cham, Switzerland, Lyon, France (Apr 23–27, 2023). [https://doi.org/10.1007/978-3-031-30617-4\\_11](https://doi.org/10.1007/978-3-031-30617-4_11)
38. Ganesh, C., Orlandi, C., Pancholi, M., Takahashi, A., Tschudi, D.: Fiat-shamir bulletproofs are non-malleable (in the algebraic group model). In: Dunkelman, O., Dziembowski, S. (eds.) *Advances in Cryptology – EUROCRYPT 2022, Part II. Lecture Notes in Computer Science*, vol. 13276, pp. 397–426. Springer, Cham, Switzerland, Trondheim, Norway (May 30 – Jun 3, 2022). [https://doi.org/10.1007/978-3-031-07085-3\\_14](https://doi.org/10.1007/978-3-031-07085-3_14)
39. Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In: Fortnow, L., Vadhan, S.P. (eds.) *43rd Annual ACM Symposium on Theory of Computing*. pp. 99–108. ACM Press, San Jose, CA, USA (Jun 6–8, 2011). <https://doi.org/10.1145/1993636.1993651>
40. Ghoshal, A., Tessaro, S.: Tight state-restoration soundness in the algebraic group model. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021, Part III. Lecture Notes in Computer Science*, vol. 12827, pp. 64–93. Springer, Cham, Switzerland, Virtual Event (Aug 16–20, 2021). [https://doi.org/10.1007/978-3-030-84252-9\\_3](https://doi.org/10.1007/978-3-030-84252-9_3)
41. Giunta, E.: On the impossibility of algebraic NIZK in pairing-free groups. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023, Part IV. Lecture Notes in Computer Science*, vol. 14084, pp. 702–730. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 20–24, 2023). [https://doi.org/10.1007/978-3-031-38551-3\\_22](https://doi.org/10.1007/978-3-031-38551-3_22)

42. Groth, J.: Simulation-sound NIZK proofs for a practical language and constant size group signatures. In: Lai, X., Chen, K. (eds.) *Advances in Cryptology – ASIACRYPT 2006*. Lecture Notes in Computer Science, vol. 4284, pp. 444–459. Springer Berlin Heidelberg, Germany, Shanghai, China (Dec 3–7, 2006). [https://doi.org/10.1007/11935230\\_29](https://doi.org/10.1007/11935230_29)
43. Groth, J.: On the size of pairing-based non-interactive arguments. In: Fischlin, M., Coron, J.S. (eds.) *Advances in Cryptology – EUROCRYPT 2016, Part II*. Lecture Notes in Computer Science, vol. 9666, pp. 305–326. Springer Berlin Heidelberg, Germany, Vienna, Austria (May 8–12, 2016). [https://doi.org/10.1007/978-3-662-49896-5\\_11](https://doi.org/10.1007/978-3-662-49896-5_11)
44. Groth, J., Ostrovsky, R., Sahai, A.: Perfect non-interactive zero knowledge for NP. In: Vaudenay, S. (ed.) *Advances in Cryptology – EUROCRYPT 2006*. Lecture Notes in Computer Science, vol. 4004, pp. 339–358. Springer Berlin Heidelberg, Germany, St. Petersburg, Russia (May 28 – Jun 1, 2006). [https://doi.org/10.1007/11761679\\_21](https://doi.org/10.1007/11761679_21)
45. Jaeger, J., Mohan, D.L.: Generic and algebraic computation models: When AGM proofs transfer to the GGM. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024, Part V*. Lecture Notes in Computer Science, vol. 14924, pp. 14–45. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). [https://doi.org/10.1007/978-3-031-68388-6\\_2](https://doi.org/10.1007/978-3-031-68388-6_2)
46. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-size commitments to polynomials and their applications. In: Abe, M. (ed.) *Advances in Cryptology – ASIACRYPT 2010*. Lecture Notes in Computer Science, vol. 6477, pp. 177–194. Springer Berlin Heidelberg, Germany, Singapore (Dec 5–9, 2010). [https://doi.org/10.1007/978-3-642-17373-8\\_11](https://doi.org/10.1007/978-3-642-17373-8_11)
47. Khoshakhlagh, H.: Revisiting simulation extractability in the updatable setting. *Cryptology ePrint Archive*, Paper 2025/2117 (2025), <https://eprint.iacr.org/2025/2117>
48. Kilian, J.: A note on efficient zero-knowledge proofs and arguments (extended abstract). In: 24th Annual ACM Symposium on Theory of Computing. pp. 723–732. ACM Press, Victoria, BC, Canada (May 4–6, 1992). <https://doi.org/10.1145/129712.129782>
49. Kohlweiss, M., Pancholi, M., Takahashi, A.: How to compile polynomial IOP into simulation-extractable SNARKs: A modular approach. In: Rothblum, G.N., Wee, H. (eds.) *TCC 2023: 21st Theory of Cryptography Conference, Part III*. Lecture Notes in Computer Science, vol. 14371, pp. 486–512. Springer, Cham, Switzerland, Taipei, Taiwan (Nov 29 – Dec 2, 2023). [https://doi.org/10.1007/978-3-031-48621-0\\_17](https://doi.org/10.1007/978-3-031-48621-0_17)
50. Kosba, A., Zhao, Z., Miller, A., Qian, Y., Chan, H., Papamanthou, C., Pass, R., Shelat, A., Shi, E.: C<sub>0</sub>C<sub>0</sub>: A framework for building composable zero-knowledge proofs. *Cryptology ePrint Archive*, Report 2015/1093 (2015), <https://eprint.iacr.org/2015/1093>
51. Lipmaa, H.: Polymath: Groth16 is not the limit. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology – CRYPTO 2024, Part X*. Lecture Notes in Computer Science, vol. 14929, pp. 170–206. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 18–22, 2024). [https://doi.org/10.1007/978-3-031-68403-6\\_6](https://doi.org/10.1007/978-3-031-68403-6_6)
52. Lysyanskaya, A., Rosenbloom, L.N.: Efficient and universally composable non-interactive zero-knowledge proofs of knowledge with security against adaptive corruptions. *Cryptology ePrint Archive*, Report 2022/1484 (2022), <https://eprint.iacr.org/2022/1484>
53. Lysyanskaya, A., Rosenbloom, L.N.: Universally composable  $\Sigma$ -protocols in the global random-oracle model. In: Kiltz, E., Vaikuntanathan, V. (eds.) *TCC 2022: 20th Theory of Cryptography Conference, Part I*. Lecture Notes in Computer Science, vol. 13747, pp. 203–233. Springer, Cham, Switzerland, Chicago, IL, USA (Nov 7–10, 2022). [https://doi.org/10.1007/978-3-031-22318-1\\_8](https://doi.org/10.1007/978-3-031-22318-1_8)
54. Maller, M., Bowe, S., Kohlweiss, M., Meiklejohn, S.: Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In: Cavallaro, L., Kinder, J., Wang, X., Katz, J. (eds.) *ACM CCS 2019: 26th Conference on Computer and Communications Security*. pp. 2111–2128. ACM Press, London, UK (Nov 11–15, 2019). <https://doi.org/10.1145/3319535.3339817>
55. Maurer, U.M.: Abstract models of computation in cryptography (invited paper). In: Smart, N.P. (ed.) 10th IMA International Conference on Cryptography and Coding. Lecture Notes in Computer Science, vol. 3796, pp. 1–12. Springer Berlin Heidelberg, Germany, Cirencester, UK (Dec 19–21, 2005). [https://doi.org/10.1007/11586821\\_1](https://doi.org/10.1007/11586821_1)
56. Micali, S.: CS proofs (extended abstracts). In: 35th Annual Symposium on Foundations of Computer Science. pp. 436–453. IEEE Computer Society Press, Santa Fe, NM, USA (Nov 20–22, 1994). <https://doi.org/10.1109/SFCS.1994.365746>
57. Micali, S.: Computationally sound proofs. *SIAM Journal on Computing* **30**(4), 1253–1298 (2000). <https://doi.org/10.1137/S0097539795284959>, <https://doi.org/10.1137/S0097539795284959>
58. National Institute of Standards and Technology: Digital Signature Standard (DSS). Tech. Rep. FIPS 186-5, National Institute of Standards and Technology (2023). <https://doi.org/10.6028/NIST.FIPS.186-5>
59. Nechaev, V.I.: Complexity of a determinate algorithm for the discrete logarithm. *Mathematical Notes* **55**(2), 165–172 (Feb 1994). <https://doi.org/10.1007/BF02113297>
60. Setty, S.: Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In: Micciancio, D., Ristenpart, T. (eds.) *Advances in Cryptology – CRYPTO 2020, Part III*. Lecture Notes in Computer Science, vol. 12172, pp.

- 704–737. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 17–21, 2020). [https://doi.org/10.1007/978-3-030-56877-1\\_25](https://doi.org/10.1007/978-3-030-56877-1_25)
61. Shoup, V.: Lower bounds for discrete logarithms and related problems. In: Fumy, W. (ed.) *Advances in Cryptology – EUROCRYPT’97*. Lecture Notes in Computer Science, vol. 1233, pp. 256–266. Springer Berlin Heidelberg, Germany, Konstanz, Germany (May 11–15, 1997). [https://doi.org/10.1007/3-540-69053-0\\_18](https://doi.org/10.1007/3-540-69053-0_18)
  62. Zhandry, M.: To label, or not to label (in generic groups). In: Dodis, Y., Shrimpton, T. (eds.) *Advances in Cryptology – CRYPTO 2022, Part III*. Lecture Notes in Computer Science, vol. 13509, pp. 66–96. Springer, Cham, Switzerland, Santa Barbara, CA, USA (Aug 15–18, 2022). [https://doi.org/10.1007/978-3-031-15982-4\\_3](https://doi.org/10.1007/978-3-031-15982-4_3)
  63. Zhang, C., Zhandry, M.: The relationship between idealized models under computationally bounded adversaries. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology – ASIACRYPT 2023, Part VI*. Lecture Notes in Computer Science, vol. 14443, pp. 390–419. Springer, Singapore, Singapore, Guangzhou, China (Dec 4–8, 2023). [https://doi.org/10.1007/978-981-99-8736-8\\_13](https://doi.org/10.1007/978-981-99-8736-8_13)
  64. Zhang, C., Zhou, H.S., Katz, J.: An analysis of the algebraic group model. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022, Part IV*. Lecture Notes in Computer Science, vol. 13794, pp. 310–322. Springer, Cham, Switzerland, Taipei, Taiwan (Dec 5–9, 2022). [https://doi.org/10.1007/978-3-031-22972-5\\_11](https://doi.org/10.1007/978-3-031-22972-5_11)



## A Additional Related Works

**UC-AGM.** Abdalla et al. [1] present a UC-AGM framework modelling the AGM behaviour in UC. However, this model fails to capture extractable non-interactive primitives such as zkSNARKs. The fundamental issue is that the UC-AGM does not require the adversary to output an explanation alongside the group elements when providing inputs to honest parties (the verifier in this case). Whereas this explanation is needed by the simulator to extract on a verify query. This asymmetry makes UC-AGM unsuitable for zkSNARKs.

**UC secure NIZKs.** Several non-succinct NIZKs have been proven to be UC secure. One common approach is via compiling a proof system in the stand-alone setting, to a UC one by essentially encrypting the witness as part of the proof. SLE can be obtained by simply decrypting this ciphertext. A series of works follow variants of this recipe [2, 3, 8, 28, 50]. Others, compile  $\Sigma$  protocols using Fischlin’s transform to achieve SLE [52, 53]. The NIZK construction from [28] was shown to be UC secure in [24]. Finally, the works of [25, 42, 44] give specific constructions of UC secure NIZKs against various notions of adaptive adversaries.

## B Preliminaries: Stand-Alone Properties

### B.1 Symbolic Shoup’s generic group model

**Figure 4:** Symbolic GGM oracle: s-G

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><u>INIT()</u></p> <ol style="list-style-type: none"> <li>1: <b>for</b> <math>j \in \{1, 2, t\}</math> : <b>do</b></li> <li>2:   <math>\tau_i \leftarrow \{\}</math></li> <li>3:   <math>\text{TAU}(i, 1)</math></li> <li>4: <b>end for</b></li> </ol> <p><u>CANONICALGEN(<math>i</math>)</u></p> <ol style="list-style-type: none"> <li>1: <b>return</b> <math>\text{TAU}(i, 1)</math></li> </ol> <p><u>OP(<math>i, (g_1, a_1), (g_2, a_2)</math>)</u></p> <ol style="list-style-type: none"> <li>1: <b>assert</b> <math>(g_1, g_2, a_1, a_2) \in S_i^2 \times \mathbb{Z}_p^2</math></li> <li>2: <b>for</b> <math>j \in \{1, 2\}</math> : <b>do</b></li> <li>3:   <math>\text{TOUCH}(i, g_j)</math></li> <li>4: <b>end for</b></li> <li>5: <math>f := a_1 \tau_i^{-1}(g_1) + a_2 \tau_i^{-1}(g_2)</math></li> <li>6: <math>h \leftarrow \text{TAU}(i, f)</math></li> <li>7: <b>return</b> <math>h</math></li> </ol> | <p><u>TOUCH(<math>i, g</math>)</u></p> <ol style="list-style-type: none"> <li>1: <b>if</b> <math>g \notin \text{img}(\tau_i)</math> <b>then</b></li> <li>2:   Initialize fresh variable <math>X</math></li> <li>3:   <math>\tau_i(X) := g</math></li> <li>4: <b>end if</b></li> </ol> <p><u>TAU(<math>i, f</math>)</u> //Internal function</p> <ol style="list-style-type: none"> <li>1: <b>if</b> <math>\tau_i(f) = \perp</math> <b>then</b></li> <li>2:   <math>\tau_i(f) \xleftarrow{\\$} S_i \setminus \text{img}(\tau_i)</math></li> <li>3: <b>end if</b></li> <li>4: <b>return</b> <math>\tau_i(f)</math></li> </ol> <p><u>PAIR(<math>g_1, g_2</math>)</u></p> <ol style="list-style-type: none"> <li>1: <b>assert</b> <math>(g_1, g_2) \in S_1 \times S_2</math></li> <li>2: <b>for</b> <math>i \in \{1, 2\}</math> <b>do</b></li> <li>3:   <math>\text{TOUCH}(i, g_i)</math></li> <li>4: <b>end for</b></li> <li>5: <math>f := \tau_1^{-1}(g_1) \cdot \tau_2^{-1}(g_2)</math></li> <li>6: <math>h \leftarrow \text{TAU}(t, f)</math></li> <li>7: <b>return</b> <math>h</math></li> </ol> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### B.2 Succinct non-interactive zero knowledge argument of knowledge

**Definition 5 (Completeness).** For all PPT adversaries  $\mathcal{A}$ , for all  $\lambda \in \mathbb{N}$ ,

$$\Pr \left[ \begin{array}{c} (i, x, w) \in \mathcal{R} \\ \implies V(\text{ivk}, x, \pi) = 1 \end{array} \middle| \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda); \\ (i, x, w) \leftarrow \mathcal{A}(\text{crs}); \\ (\text{ipk}, \text{ivk}) \leftarrow I(i, \text{crs}); \\ \pi \leftarrow P(\text{ipk}, x, w) \end{array} \right] = 1.$$



All algorithms  $(\mathcal{A}, \text{Setup}, I, P, V)$  have oracle access to  $\text{RO}, \mathcal{G}$ .

**Definition 6 (Non-Interactive Zero Knowledge).** Let  $\Pi = (\text{Setup}, I, P, V)$  be a non-interactive argument of knowledge for relation  $\mathcal{R}$ .  $\Pi$  satisfies **NIZK** in the non-programmable random oracle and generic group model, if there exists an efficient simulator  $S$  such that for all PPT distinguishers  $\mathcal{A}$  the following advantage is negligible in  $\lambda$ ,

$$\text{Adv}_{\mathcal{A}}^{\text{NIZK}}(\lambda) := \left| \Pr [\text{NIZK-0}_{\mathcal{A}}(1^\lambda) = 1] - \Pr [\text{NIZK-1}_{\mathcal{A}}(1^\lambda) = 1] \right|$$

where **NIZK-0** and **NIZK-1** are defined in Fig. 5.

**Figure 5:** Security game for NIZK

NIZK-0 $_{\mathcal{A}}(1^\lambda)$

- 1:  $\text{pp} \leftarrow \text{Gen}(1^\lambda)$
- 2:  $(\text{crs}, \text{td}) \leftarrow \text{Setup}(\text{pp})$
- 3:  $b \leftarrow \mathcal{A}^{\text{RO}, \mathcal{G}, P, V}(\text{crs})$
- 4: **return**  $b$

NIZK-1 $_{\mathcal{A}}(1^\lambda)$

- 1:  $\text{pp} \leftarrow \text{Gen}(1^\lambda)$
- 2:  $(\text{crs}, \text{td}) \leftarrow \text{Setup}(\text{pp})$
- 3:  $\mathcal{Q}_S := \{\}$
- 4:  $\text{st} := (\text{td}, \mathcal{Q}_S)$
- 5:  $b \leftarrow \mathcal{A}^{\text{RO}, \mathcal{G}, S_1, V}(\text{crs})$
- 6: **return**  $b$

$S_1(x, w)$

- 1: **assert**  $(x, w) \in \mathcal{R}$
- 2:  $(\pi, \text{st}) \leftarrow S(\text{st}, \text{crs}, x)$
- 3:  $\mathcal{Q}_S \leftarrow \mathcal{Q}_S \cup (x, \pi)$
- 4: **return**  $\pi$

### B.3 Computing group representation

**Algorithm 1:** FINDREP

FINDREP( $i, h, B, \text{tr}$ )

- 1: **assert**  $i \in \{1, 2\}$
- 2: Parse  $B := (B_1, \dots, B_n)$
- 3: Initially  $\text{Rep}[h] = 0^n, \forall h \in S_i$
- 4: Initialize  $\text{Rep}[B_j][j] := 1$ , for  $j \in [n]$
- 5: **for**  $op = (\text{OP}, i, g_1, g_2, a_1, a_2, h) \in \text{tr}_{\mathcal{G}}$  **do**:
- 6:      $\text{Rep}[h] := a_1 \cdot \text{Rep}[g_1] + a_2 \cdot \text{Rep}[g_2]$
- 7: **end for**
- 8: **return**  $\text{Rep}[h]$

FINDREP( $t, h, B, \text{tr}$ )

- 1: Parse  $B = (B^{(1)}, B^{(2)}, B^{(t)})$
- 2: **for**  $i \in \{1, 2\}$  **do**,
- 3:     Parse  $B^{(i)} = (B_1^{(i)}, \dots, B_{n_i}^{(i)})$
- 4:     Initially  $\text{Rep}_i[h] \leftarrow 0^{n_i}$
- 5:     Initialize  $\text{Rep}[B_j^{(i)}][j] := 1$ , for  $j \in [n_i]$
- 6: **end for**
- 7: Parse  $B^{(t)} = (B_1^{(t)}, \dots, B_{n_t}^{(t)})$
- 8: Initially  $\text{Rep}_t[h] \leftarrow 0^{n_t + n_1 \cdot n_2}$
- 9: Initialize  $\text{Rep}[B_j^{(i)}][j] := 1$ , for  $j \in [n_t + n_1 \cdot n_2]$
- 10: **for**  $op \in \text{tr}$  **do**
- 11:     **if**  $op = (\text{OP}, i, g_1, g_2, a_1, a_2, h) \in \text{tr}$  **then**
- 12:          $\text{Rep}[h] := a_1 \cdot \text{Rep}[g_1] + a_2 \cdot \text{Rep}[g_2]$
- 13:     **end if**
- 14:     **if**  $op = (\text{PAIR}, g_1, g_2, h) \in \text{tr}$  **then**
- 15:          $\text{Rep}_t[h] \leftarrow (0^{n_t}, \text{Rep}_1[g_1] \otimes \text{Rep}_2[g_2])$
- 16:      $// c \otimes d := (c_k \cdot d_\ell)_{k \in [n_1], \ell \in [n_2]}$
- 17:     **end if**
- 18: **end for**
- 19: **return**  $\text{Rep}_t[h]$

## C Preliminaries: Universal Composability

**Syntax for ideal functionalities and protocols.** We give a brief explanation of the execution model in the UC framework. For an more in depth discussion, we refer the reader to [13, 21].

As [13] we will follow the UCGS [6] formalism. In both worlds, in a session  $sid$ , the environment  $\mathcal{Z}$  initializes a number of honest parties and an adversary  $\mathcal{A}$  (i.e. static corruption). It provides inputs to all, reads outputs of all parties, and has full access to the view of  $\mathcal{A}$ .  $\mathcal{G}$  is a global functionality available to all parties,  $\mathcal{A}$  and the environment  $\mathcal{Z}$ , and can be directly queried.

**Real World:** The execution takes place in the  $(\mathcal{F}_{\text{setup}}, \mathcal{G})$ -hybrid model.  $\mathcal{F}_{\text{setup}}$  is a local functionality available only to parties within the session, while  $\mathcal{G}$  is a global functionality as above. During the execution,  $\mathcal{Z}$  can issue PROVE, VERIFY commands. On receiving  $(\text{PROVE}_{sid}, x, w)$ , an honest party executes the SNARK proving algorithm  $P$ , and on receiving  $(\text{VERIFY}_{sid}, x, \pi)$ , an honest verifier executes the SNARK verification algorithm  $V$ . In addition to this,  $\mathcal{Z}$  can query  $\mathcal{G}$  on OBSERVE to receive the set of illegal queries.

**Ideal World:** In this experiment, honest parties in  $sid$  forward their inputs to  $\mathcal{F}_{\text{nizk}}$ . Access to  $\mathcal{A}$ ,  $\mathcal{G}$  remains the same in both worlds.

$\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}}(\lambda, z)$  denotes the distribution of  $\mathcal{Z}$ 's binary output after an execution  $\Pi$  in presence of  $\mathcal{A}$  and given access to (local or global) functionality  $\mathcal{F}$ , where  $\lambda \in \mathbb{N}$  is the security parameter and  $z \in \{0, 1\}^*$  is an auxiliary input to  $\mathcal{Z}$ .  $\{\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{Z}}^{\mathcal{F}}(\lambda, z)\}_{\lambda \in \mathbb{N}, z \in \{0, 1\}^*}$  denotes the ensemble of such random variables.

### Functionality 3: $\mathcal{F}_{\text{setup}}$

(Parametrized by the algorithm *Setup* of a NIZK  $\Pi$ .)

INIT<sub>sid</sub>( )

1: **for**  $i \in \{1, 2\}$  **do**:

2:    $g_{i, sid} \xleftarrow{\$} S_i$

3:   TOUCH<sub>sid</sub>( $g_{i, sid}$ )

4: **end for**

5:  $(\text{crs}, \text{td}) \leftarrow \text{Setup}(g_{1, sid}, g_{2, sid})$

GETCRS<sub>sid</sub>( )

1: **return**  $\text{crs}$

### Functionality 4: $\mathcal{G}_{\text{RO}}$

$\mathcal{G}_{\text{RO}}$  is parametrized by the output length  $\ell(\lambda)$ .

QUERY: Upon receiving a query  $(\text{QUERY}, x)$ , from some party  $\mathcal{P} = (pid, sid)$  or from the adversary do:

1: Look up  $v$  if there is a pair  $(x, v)$  for some  $v \in \{0, 1\}^{\ell(\lambda)}$  in the (initially empty) list  $\mathcal{Q}_{\text{RO}}$  of past queries. Else, choose uniformly  $v \in \{0, 1\}^{\ell(\lambda)}$  and store the pair  $(x, v)$  in  $\mathcal{Q}_{\text{RO}}$ .

2: Parse  $x$  as  $(s, x')$ . If  $sid \neq s$  then add  $(s, x', v)$  to the (initially empty) observable set  $Ob_{\text{RO}}$

3: Return  $v$  to  $\mathcal{P}$ .

OBSERVE: Upon receiving a request  $(\text{OBSERVE}, sid)$ , return the list  $Ob_{\text{RO}}$  restricted to tuples with  $sid$ .

### Functionality 5: $\mathcal{G}$

Parameters of the functionality:

- A prime number  $p$
- Sets  $S_1, S_2, S_t$  with  $|S_i| = p$  for all  $i \in \{1, 2, t\}$ .

Internal state maintained by the functionality:

- $\tau_i$ : Random encoding  $\mathbb{Z}_p \rightarrow S_i$ , for  $i \in \{1, 2, t\}$ .
- $X_{i,sid}$ : Initially empty list. This will record formal variables defined for session  $sid$ .  $X_{sid}$  denotes the concatenation  $X_{1,sid} : X_{2,sid} : X_{t,sid}$ .
- $R_i$ : Initially empty sets of polynomial. For a group element  $g \in S_i$ ,  $R_i[g]$  will record the set of polynomial representation of  $g$ .
- $Ob$ : Initially empty list. This will record the set of observable group operation queries.

Furthermore:

- A polynomial  $f$  is in  $\text{Legal}_{sid}^{\text{OP}}$  if it only contains variables in  $X_{sid}$  and a zero constant term.
- A polynomial  $f$  is in  $\text{Legal}_{sid}^{\text{PAIR}}$  if in addition to constraints in  $\text{Legal}_{sid}^{\text{OP}}$ , for all degree 1 terms with variable  $X$ ,  $X \in X_{t,sid}$ .

INIT<sub>sid</sub>()

```

1: for  $i \in \{1, 2, t\}$  : do
2:    $\tau_i \xleftarrow{\$} \text{INJ}(\mathbb{Z}_p, S_i)$ 
3:    $R_i[\tau_i(1)] \leftarrow \{1\}$ 
4: end for
```

CANONICALGEN<sub>sid</sub>( $i$ )

```

1: return  $\tau_i(1)$ 
```

OBSERVE<sub>sid</sub>

```

1: return  $Ob$ 
```

OP<sub>sid</sub>( $i, (g_1, a_1), (g_2, a_2)$ )

```

1: assert  $(g_1, g_2, a_1, a_2) \in S_i^2 \times \mathbb{Z}_p^2$ 
2: for  $j \in \{1, 2\}$  do
3:   TOUCHsid( $i, g_j$ )
4: end for
5:  $h \leftarrow \tau_i(a_1 \tau_i^{-1}(g_1) + a_2 \tau_i^{-1}(g_2))$ 
6:  $R_i[h] \leftarrow R_i[h] \cup (a_1 R_i[g_1] + a_2 R_i[g_2])$ 
7: if  $\exists f \in R_i[h] : f \notin \text{Legal}_{sid}$  then
8:    $Ob \leftarrow Ob \cup \{(OP, i, g_1, g_2, a_1, a_2, h)\}$ 
9: end if
10: return  $h$ 
```

TOUCH<sub>sid</sub>( $i, g$ )

```

1: if  $R_i[g] = \emptyset$  then
2:   Initialize fresh variable  $X$ 
3:    $X_{i,sid} \leftarrow X_{i,sid} : [X]$ 
4:    $R_i[g] \leftarrow \{X\}$ 
5: end if
```

PAIR<sub>sid</sub>( $g_1, g_2$ )

```

1: assert  $(g_1, g_2) \in S_1 \times S_2$ 
2: for  $i \in \{1, 2\}$  do
3:   TOUCHsid( $i, g_i$ )
4: end for
5:  $h \leftarrow \tau_t(\tau_1^{-1}(g_1) \cdot \tau_2^{-1}(g_2))$ 
6:  $R_t[h] \leftarrow R_t[h] \cup (R_1[g_1] \cdot R_2[g_2])$ 
7: if  $\exists f \in R_t[h] : f \notin \text{Legal}_{sid}$  then
8:    $Ob \leftarrow Ob \cup \{(PAIR, g_1, g_2, h)\}$ 
9: end if
10: return  $h$ 
```

## D UC Simulator and Adversary's Definition

### Algorithm 2: Simulator $\mathcal{S}$

|                                                         |                                                                                  |
|---------------------------------------------------------|----------------------------------------------------------------------------------|
| <u>INIT<sub>sid</sub>()</u>                             | <u><math>\mathcal{G}_{\text{RO}}.\text{QUERY}_{sid}()</math></u>                 |
| 1: Run the code of $\mathcal{F}_{\text{setup}}$         | 1: Forward the query to $\mathcal{G}_{\text{RO}}$ .                              |
| 2: Store $\text{td}, \text{crs}$                        | <u><math>\mathcal{G}_{\text{RO}}.\text{OBSERVE}_{sid}()</math></u>               |
| 3: Initialize $\text{st} := (\text{td}, \mathcal{Q}_S)$ | 1: Forward the query to $\mathcal{G}_{\text{RO}}$ .                              |
| 4: Initialise empty set $Ob_S$                          | <u><math>\mathcal{S}.\text{SIMULATE}_{sid}(x)</math></u>                         |
| <u>GETCRS<sub>sid</sub>()</u>                           | 1: $\pi \leftarrow S(\text{st}, \text{crs}, x)$                                  |
| 1: Return $\text{crs}$                                  | 2: $\mathcal{Q}_S \leftarrow \mathcal{Q}_S : (x, \pi)$                           |
| <u>CANONICALGEN<sub>sid</sub>()</u>                     | <u><math>\mathcal{S}.\text{EXTRACT}_{sid}(x, \pi)</math></u>                     |
| 1: Forward the query to $\mathcal{G}$ .                 | 1: $b \leftarrow V^{\mathcal{G}_{\text{RO}}, \mathcal{G}}(\text{crs}, x, \pi)$   |
| <u>OBSERVE<sub>sid</sub>()</u>                          | 2: If $b = 0$ , return $\perp$                                                   |
| 1: Forward the query to $\mathcal{G}$ .                 | 3: $Ob_{sid} \leftarrow \text{OBSERVE}_{sid}$                                    |
| <u>OP<sub>sid</sub>()</u>                               | 4: $Ob_S := Ob_{sid} \cup Ob_S$                                                  |
| 1: Forward the query to $\mathcal{G}$ .                 | 5: <b>for</b> $i \in \{1, 2, t\}$ <b>do</b> ,                                    |
| 2: Update $Ob_S$                                        | 6: $\{(i, h_j, B_j)\}_{j \in k_i} \leftarrow E_0(\text{crs}, x, \pi)$            |
| <u>TOUCH<sub>sid</sub>()</u>                            | 7: <b>for</b> $j \in k_i$ <b>do</b>                                              |
| 1: Forward the query to $\mathcal{G}$ .                 | 8: $\mathbf{c}^{(i)}[j] \leftarrow \text{FINDREP}(i, h_j, B_j, Ob_S)$            |
| <u>PAIR<sub>sid</sub>()</u>                             | 9: <b>end for</b>                                                                |
| 1: Forward the query to $\mathcal{G}$ .                 | 10: <b>end for</b>                                                               |
| 2: Update $Ob_S$                                        | 11: Run $w \leftarrow E_1(\mathbf{c}^{(1)}, \mathbf{c}^{(2)}, \mathbf{c}^{(t)})$ |
|                                                         | 12: <b>return</b> $w$                                                            |

**Notations.** Recall that **s-G** refers to the symbolic version of shoup's GGM. For common data-structures (for example  $\tau$ ) that would be maintained both by the adversary and **s-G**, we denote the former simply as  $\tau$ , while **s-G**. $\tau$  for the latter. We use,  $X_\pi$  denote the set of variables associated with group elements that are output by either the **PROVE** command or  $\text{INIT}_{sid}$  execution.  $X_{sid}$  and  $X_\pi$  together contain all variables belonging to session  $sid$ . When we write  $\mathbf{f}^{(3)}(\tau(X_{sid'}))$ , it is a short hand to mean that each  $X$  in  $\mathbf{f}$  should be replaced by corresponding group element, and sequence of group operation should be performed according to the definition of  $\mathbf{f}$ .

$\mathcal{A}^*(\mathcal{Z})$ : Run the environment  $\mathcal{Z}$  answering the queries it requests as follows:

1.  $\mathcal{G}_{\text{RO}}$  queries:

- **QUERY**: Parse the query as  $(sid^*, x)$ . If  $sid^* = sid$ , then forward the query  $x$  to standard **RO** in the underlying game. Otherwise, simulate a response by lazy sampling. If the query was made in a session  $\neq sid^*$ , record it in the observable set  $Ob_{\text{RO}}$ .
- **OBSERVE**: Return  $Ob_{\text{RO}}$ .

2. **s-G** queries:

- **INIT**(): To initialize **s-G** functionality,
  - (a) Invoke **s-G**.**INIT**()
  - (b) For  $i \in \{1, 2, t\}$ , call **s-G**.**CANONICALGEN**( $i$ ) to obtain **s-G**. $\tau_i(1)$  and initialize  $\tau_i(1) := \text{s-G}.\tau_i(1)$ .
  - (c) For  $i \in \{1, 2, n\}$ ,  $g \xleftarrow{\$} S_i$  and query **s-G**.**OP**( $i, g, g, 1, -1$ ) to receive  $h_i$ . Initialize  $\tau_i(0) := h_i$ .

- (d) Initialize empty lists for  $X_{i,sid}, X_{i,\pi}, X_{i,sid'}$  Ob.
- CANONICALGEN<sub>sid</sub>(i): Forward query to **s-G.CANONICALGEN**(i), and return the result.
- OBSERVE<sub>sid</sub>(): Return the set Ob.
- OP<sub>sid</sub>(i, g<sub>1</sub>, g<sub>2</sub>, a<sub>1</sub>, a<sub>2</sub>):
  - (a) Query to **s-G.OP**(i, g<sub>1</sub>, g<sub>2</sub>, a<sub>1</sub>, a<sub>2</sub>) and receive  $h$  in return. If  $h = \perp$  then return  $\perp$ . Else, continue.
  - (b) For each  $j \in \{1, 2\}$ , execute the code in **TOUCH<sub>sid</sub>(i, g<sub>j</sub>)**.
  - (c) For each  $j \in \{1, 2\}$ , retrieve the polynomial for  $g_j$ :  $p_{g_j}(X_{i,sid}, X_{i,\pi}, X_{i,sid'}) \leftarrow \tau_i^{-1}(g_j)$  such that  $p_{g_j}$  is a linear polynomial.
  - (d) Compute the result polynomial  $f(X_{i,sid}, X_{i,sid'}, X_{i,\pi}) \leftarrow a_1 p_{g_1}(X_{i,sid}, X_{i,\pi}, X_{i,sid'}) + a_2 p_{g_2}(X_{i,sid}, X_{i,\pi}, X_{i,sid'})$ .
  - (e) Update  $\tau_i$  with respect to  $f$ : if  $h \notin \text{im}(\tau_i)$ , then define  $\tau_i(f) := h$ .
  - (f) Rewrite  $f := f^{(1)}(X_{i,sid}, X_{i,\pi}) + f^{(2)}(X_{i,sid'}) + c$ , where both  $f^{(1)}, f^{(2)}$  are linear polynomials with zero constant term, and  $c$  is a constant.
  - (g) Update Ob: Locally check if  $c = 0$  and  $f^{(2)}$  is a zero polynomial. If not then update  $Ob \leftarrow Ob : [(OP, i, g_1, g_2, a_1, a_2, h)]$ .
- OP<sub>sid'</sub>(i, g<sub>1</sub>, g<sub>2</sub>, a<sub>1</sub>, a<sub>2</sub>): Behave identically as **OP<sub>sid</sub>** until the last step. To update Ob do the following,
  - (a) Let  $f^{(1)}(X_{i,sid}, X_{i,\pi}) := \langle \mathbf{a}, X_{i,sid} \rangle + \langle \mathbf{b}, X_{i,\pi} \rangle$ . Compute  $h \leftarrow \langle \mathbf{a}, \tau_i(X_{i,sid}) \rangle + \langle \mathbf{b}, \tau_i(X_{i,\pi}) \rangle$  by querying **s-G.OP**(i, ·).
  - (b) If  $c = 0$  and  $h \neq \tau_i(0)$  then update  $Ob \leftarrow Ob : [(OP, i, g_1, g_2, a_1, a_2, h)]$ .
- PAIR<sub>sid</sub>(g<sub>1</sub>, g<sub>2</sub>):
  - (a) Query to **s-G.PAIR**(g<sub>1</sub>, g<sub>2</sub>) and receive  $h$  in return. If  $h = \perp$  then return  $\perp$ . Else, continue.
  - (b) For each  $i \in \{1, 2\}$ , execute the code in **TOUCH<sub>sid</sub>(i, g<sub>i</sub>)**.
  - (c) Retrieve the polynomial for  $g_i$ :  $p_{g_i}(X_{sid}, X_{\pi}, X_{sid'}) \leftarrow \mathbf{s-G}.\tau_i^{-1}(g_i)$  such that  $p_{g_i}$  is a linear polynomial with the constant term denoted by  $c_i$ .
  - (d) Compute the result polynomial  $f(X_{sid}, X_{sid'}, X_{\pi}) \leftarrow p_{g_1}(X_{sid}, X_{\pi}, X_{sid'}) \cdot p_{g_2}(X_{sid}, X_{\pi}, X_{sid'})$ .
  - (e) Update  $\mathbf{s-G}.\tau_t$  with respect to  $f$ : if  $h \notin \text{im}(\mathbf{s-G}.\tau_t)$ , define  $\mathbf{s-G}.\tau_t(f) := h$ .
  - (f) Check if  $c_1, c_2 = 0$ . If not, update  $Ob \leftarrow Ob : [(PAIR, g_1, g_2, h)]$ . Else, continue.
  - (g) Rewrite  $f := f^{(1)}(X_{sid}, X_{\pi}) + f^{(2)}(X_{sid}, X_{\pi}, X_{sid'})$ , where  $f^{(2)} := \langle \mathbf{v}_1, X_{sid} \circ X_{sid'} \rangle + \langle \mathbf{v}_2, X_{\pi} \circ X_{sid'} \rangle + \langle \mathbf{v}_3, X_{sid'} \circ X_{sid'} \rangle$  for some vector of field elements  $\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$ .
  - (h) Rewrite the terms  $\langle \mathbf{v}_1, X_{sid} \circ X_{sid'} \rangle + \langle \mathbf{v}_2, X_{\pi} \circ X_{sid'} \rangle$  as  $f^{(3)}(X_{sid'}) \cdot (f^{(4)}(X_{sid}) + f^{(5)}(X_{\pi}))$ .
  - (i) Do the following checks:
    - Compute  $h \leftarrow f^{(3)}(\tau(X_{sid'})) \cdot (f^{(4)}(\tau(X_{sid})) + f^{(5)}(\tau(X_{\pi})))$  by querying **s-G.OP** and **s-G.PAIR** appropriately.
    - If  $h \neq \tau_t(0)$ , then update  $Ob \leftarrow Ob : [(PAIR, g_1, g_2, h)]$ .
- PAIR<sub>sid'</sub>(g<sub>1</sub>, g<sub>2</sub>): Behave identically as **PAIR<sub>sid</sub>** until (and including) rewriting  $f$ . Then do the following,
  - (a) Do the following checks:

- Compute  $h \leftarrow f^{(1)}(\tau(X_{sid}), \tau(X_\pi))$  by querying  $\mathbf{s}\text{-G.OP}$  and  $\mathbf{s}\text{-G.PAIR}$  appropriately. Check if  $h = \tau_t(0)$ .
  - Check term  $\langle v_1, X_{sid} \circ X_{sid'} \rangle + \langle v_2, X_\pi \circ X_{sid'} \rangle$  as in  $\text{PAIR}_{sid}$  case.
- (b) Update  $Ob$ : If either check above fails then update  $Ob \leftarrow Ob : [(PAIR, g_1, g_2, h)]$ .
- $\text{TOUCH}_{sid}(i, g)$ :
- (a) Forward query to  $\mathbf{s}\text{-G}$ .
- (b) If  $g \notin \text{im}(\tau_i)$  then,
- i. Initialize a fresh variable  $X$  and  $X_{i,sid} \leftarrow X_{i,sid} : [X]$
  - ii.  $\tau_i(X) \leftarrow g$
3.  $\text{INIT}_{sid}()$ :
- (a) Forward the query to  $\text{Setup}$  interface of the game and receive  $\text{crs}$  in return.
- (b) Store  $\text{crs}$  as  $\text{crs}_{sid}$ .
- (c) For all group elements  $g \in \text{crs}_{sid}$ , execute the code in  $\text{TOUCH}_{sid}$  adding the fresh variable to  $X_\pi$  and define  $\tau(X) := g$ .
4.  $\text{GETCRS}_{sid}()$ : Output  $\text{crs}_{sid}$ .
5.  $\text{PROVE}_{sid}(x, w)$ :
- (a) Forward the query to  $P$  interface of the game and receive proof string  $\pi$  as the output.
- (b) For all group elements  $g \in \pi$  such that  $g \notin \text{im}(\tau)$ , execute the code in  $\text{TOUCH}_{sid}$  adding the fresh variable to  $X_\pi$  and define  $\tau(X) := g$ .
6.  $\text{VERIFY}_{sid}$ : Forward the query to  $V$  interface of the game and return the decision bit  $b$ .

## E Omitted Claims for Theorem 4

**Claim 3.**  $\text{Hyb}_0$  and  $\text{Hyb}_1$  are indistinguishable.

*Proof.* This is implied by Lemma 3 in [13].

**Claim 4.**  $\text{Hyb}_1$  and  $\text{Hyb}_2$  are indistinguishable.

*Proof.* In  $\text{Hyb}_1$   $\mathcal{B}$  behaves exactly as honest parties would in the real world. In addition to this, it intercepts  $\mathcal{F}_{\text{setup}}$  and  $\mathbf{s}\text{-G}$  queries without tampering them in any way. Hence, the view of  $\mathcal{Z}$  is identical in the two hybrids.

**Claim 5.**  $\text{Hyb}_2$  and  $\text{Hyb}_3$  are indistinguishable.

*Proof.* An additional abort condition is introduced in  $\text{Hyb}_3$  compared to  $\text{Hyb}_2$ , and otherwise the two hybrids are identical. This event happens when at time  $t$ ,  $\mathcal{Z}$  makes a group query containing a group element  $g$  such that  $g$  is not yet in the view of  $\mathcal{Z}$ , and  $g$  has already appeared previously as a part of an honest algorithm execution (as a result of an intermediate group operation). Without having seen  $g$  before (either as a direct result of querying interfaces in  $sid$ , or as an output of a group query made by  $\mathcal{Z}$ ), the probability of this event boils down to  $\mathcal{Z}$  guessing  $g$  correctly. Assuming that whenever an honest algorithm samples group elements, it does so with high min-entropy, this event happens with negligible probability.

**Claim 6.**  $\text{Hyb}_3$  and  $\text{Hyb}_4$  are indistinguishable.



*Proof.* When the constant term in the resultant polynomial is non zero, the tuple is always added to *Ob*. Also, for PAIR queries if any monomial with degree 1 has a variable not corresponding to the target group, then this too is added to *Ob*. Hence the differences between the hybrid are merely syntactic and otherwise they are identical.

**Claim 7.** *Hyb<sub>4</sub> and Hyb<sub>5</sub> are indistinguishable.*

*Proof.* The difference between the two hybrids lies in how observability is decided for OP queries made in the challenge session *sid*. Since the polynomials are linear, the two hybrids are identical: In Hyb<sub>4</sub>, receiving a query for  $\text{OP}_{sid}(i, g_1, g_2, a_1, a_2)$  the group functionality computes the polynomial representation of the result  $(a_1\tau_i(g_1) + a_2\tau(g_2))$ , then checks if this is a linear polynomial of the form  $\sum_{X \in X_{sid}} X \cdot \mathbb{Z}_p[X_{sid}]$  (the constant term is zero because of the previous hybrid). This is equivalent to checking if the coefficients corresponding to variables in  $X_{sid'}$  are zero (i.e., checking if  $f^{(2)}$  is zero) in Hyb<sub>5</sub>.

**Claim 8.** *Hyb<sub>5</sub> and Hyb<sub>6</sub> are indistinguishable.*

*Proof.* The difference between the two hybrids lies in how observability is decided for OP queries made in session *sid'*. As in the previous claim, in the first hybrid the group functionality checks if the resultant polynomial is of the form  $\sum_{X \in X_{sid'}} X \cdot \mathbb{Z}_p[X_{sid'}]$ , which is equivalent to checking that the coefficients corresponding to variables in  $X_{sid}$  are zero (i.e., checking if  $f^{(1)}$  is zero).

**Claim 9.** *Hyb<sub>6</sub> and Hyb<sub>7</sub> are indistinguishable.*

*Proof.* The difference between the two hybrids is in how observability is decided for PAIR queries made in the challenge session *sid*. The first hybrid, determines legality by looking at the full polynomial, while in the second hybrid only term  $f^{(2)}$  is checked. It is easy to see that this term contains possible illegal monomials (degree 2  $X_{sid'}$  terms and cross terms). Hence the checks are identical in both hybrids.

**Claim 10.** *Hyb<sub>7</sub> and Hyb<sub>8</sub> are indistinguishable.*

*Proof.* The difference between the two hybrids is in how observability is decided for PAIR queries made in the session *sid'*. Term  $f^{(1)} + \langle \mathbf{v}_1, X_{sid} \cdot X_{sid'} \rangle$  holds all possible illegal monomials. Hence the checks in both the hybrids are identical.

**Claim 11.** *Hyb<sub>8</sub> and Hyb<sub>9</sub> are indistinguishable.*

*Proof.* The difference in Hyb<sub>9</sub> is that instead of forwarding calls to  $\mathcal{F}_{\text{setup}}$ ,  $\mathcal{B}$  runs the code internally and responds to the queries made to  $\mathcal{F}_{\text{setup}}$  itself.  $\mathcal{B}$  behaves identically as  $\mathcal{F}_{\text{setup}}$  except that now it maintains the trapdoors internally. However, the view for  $\mathcal{A}$  and  $\mathcal{Z}$  still remains identical.

## F AGM to GGM Lifting Theorem

### F.1 Lifting Result

We will now present our main lifting result, establishing the relationship between algebraic adversaries and the type-safe framework in the presence of random oracles. Most of the definitions, theorem and proofs remain largely unchanged. The big take-away of this section is that appropriate versions of random oracles can be compatible with type-safe models while preserving security guarantees. This allows to capture SNARKs in this framework.

**Advantage Metrics.** Let  $\text{Ch}$  denote a security game in  $\text{SA}^{\text{RO}_{\text{ts}}}[U]$ .  $\text{Ch}$  is given the security parameter as the input and interacts with an adversarial algorithm  $A = (A_1, \dots, A_k)$ , where each  $A_i$  passes the entire state to the next. Both  $\text{Ch}$  and  $A$  have access to  $\text{RO}_{\text{ts}}$ . After interaction,  $\text{Ch}$  outputs a bit  $b$ . The probability of  $b = 1$  after the interaction between  $\text{Ch}$  and  $A$  is denoted by  $\text{Adv}_U^{\text{Ch}}(A)$ . The game  $\text{Ch}$  is said to be generically hard if  $\text{Adv}_U^{\text{Ch}}(A) \leq \text{negl}(\lambda)$  for all PPT  $A \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$ .

**Reduction.** Fix  $U$  and  $\text{RO}_{\text{ts}}$ . An algebraic reduction is a map  $R : \text{AA}^{\text{RO}_{\text{ts}}}[U] \rightarrow \text{AA}^{\text{RO}_{\text{ts}}}[U]$ . It is model preserving if  $A \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$  implies  $R[A] \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$ . It is efficiency preserving if  $R[A]$  is PPT algorithm whenever  $A$  is.

**AGM-reduction.** Fix  $U$  and  $\text{RO}_{\text{ts}}$ . Fix advantage metrics  $A_U^{\text{met}}$  and  $\text{Adv}_U^{\text{met}_1}, \dots, \text{Adv}_U^{\text{met}_n}$ , for a constant  $n$ . We say algebraic reductions  $R_1, \dots, R_n$  are  $\Delta_i$ -AGM reductions for these metrics if for all  $A \in \text{AA}_U^{\text{RO}_{\text{ts}}}$ , we have,

$$\text{Adv}_U^{\text{met}}(A) \leq \Delta_1 \cdot \text{Adv}_U^{\text{met}_1}(R_1[A]) + \dots \Delta_n \cdot \text{Adv}_U^{\text{met}_n}(R_n[A])$$

**Theorem 8.** Fix universe  $U$  and advantage metrics  $\text{Adv}_U^{\text{met}}, \text{Adv}_U^{\text{met}_1}, \dots, \text{Adv}_U^{\text{met}_n}$ , where  $\text{Adv}_U^{\text{met}_1}, \dots, \text{Adv}_U^{\text{met}_n}$  are generically hard. Let  $R_1, \dots, R_n$  be  $\Delta_i$ -AGM reductions for these metrics that are model preserving and efficiency preserving. Then  $\text{Adv}_U^{\text{met}}$  is generically hard.

*Proof.* Let  $A \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$  be given against security game corresponding to  $\text{Adv}_U^{\text{met}}$ . Because all  $R_i$ 's are  $\Delta_i$ -AGM reductions then,

$$\text{Adv}_U^{\text{met}}(A) \leq \Delta_1 \cdot \text{Adv}_U^{\text{met}_1}(R_1[A]) + \dots \Delta_n \cdot \text{Adv}_U^{\text{met}_n}(R_n[A])$$

Since each  $R_i$  is also model preserving and efficiency preserving,  $R_i[A]$  is efficient and in  $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  whenever  $A$  is. Finally, since security games corresponding to  $\text{Adv}_U^{\text{met}_1}, \dots, \text{Adv}_U^{\text{met}_n}$  are generically hard, each  $\Delta_i \cdot \text{Adv}_U^{\text{met}_i}(R_i[A])$  must be negligible. Since  $n$  is a constant  $\text{Adv}_U^{\text{met}}(A)$  is upper-bounded by a negligible function.

## G Equivalence of Type Safe model and Random Representation model with Access to a Type Safe Oracles

To stick with standard GGM notation, we are adding a labelling function  $\text{label} : \mathbb{Z}_p \rightarrow S$  to the universe of computation. In the type safe model this function gives you an  $\langle \text{elem} \rangle$ . For example, an algorithm in  $\text{TSA}[U]$  would execute  $\text{label}(a)$  by calling  $\mathcal{O}_f(\langle \text{elem} \rangle g, \perp, \langle \text{bits} \rangle a, \perp)$ , where  $g$  is a group generator.

We define  $\text{RR}[U_{\hat{\sigma}}]$  as the set of algorithms with access to an oracle that takes bitstrings labels  $\ell_1, \ell_2$  as inputs (matching the definition of Figure 1), this is equivalent to set of algorithms with access to a randomized encoding  $\hat{\sigma}$  (encode function) and  $\hat{\sigma}^{-1}$  (decode function) that would call the group oracle as  $\sigma(\mathcal{O}_f(\langle \text{elem} \rangle (\sigma^{-1}(\langle \text{bits} \rangle \ell_1)), \langle \text{elem} \rangle (\sigma^{-1}(\langle \text{bits} \rangle \ell_2)), \langle \text{bits} \rangle a, \langle \text{bits} \rangle b))$  and defined as  $\text{SA}[U_{\hat{\sigma}}]$  following Jaeger Mohan hierarchy.

**Definition 7 (Strong and weak canonical translation from  $\text{TSA}^{\text{RO}_{\text{ts}}}$  to  $\text{RR}^{\text{RO}}$ ).** Let  $U := (S, \Pi, \mathcal{R}, \hat{\sigma})$  be a universe of computation, and let  $\text{RO}_{\text{ts}}, \text{RO}$  be (typed)-random oracles. Let  $\mathcal{A} \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$  be an algorithm described in the pseudocode model. The canonical translation of  $\mathcal{A}$  in the  $\text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$  model is defined as the algorithm:

- Run  $\ell \leftarrow \hat{\sigma}(e)$  for all element variables  $e$ , they are therefore replaced by their bitstring encoding.
- Labelling and group operation using the type safe oracle are replaced by running the labelling and group operation on the corresponding labels with the GGM oracle.

- Equality function calls are preplaced by bitstrings equality test.

**Strong.** Call to the oracle  $\text{RO}_{\text{ts}}((e_i, s_i)_{i \in [n]})$  are replaced by the corresponding labels prefixed with a 0 bit. Bit strings are prefixed with a 1 bit and there size. Then call RO on the concatenated string  $0\|e_1\|1\|\text{len}(s_1)\|s_1\| \dots \|0\|e_n\|1\|\text{len}(s_n)\|s_n$ .

**Weak.** Call to the oracle  $\text{RO}_{\text{ts}}((e_i, s_i)_{i \in [n]})$  are replaced by the corresponding labels. Then call RO on the concatenated string  $\|e_1\|\|s_1\| \dots \|e_n\|\|s_n$ .

*Remark 1.* The difference between strong and weak translation might seems arbitrary as one might thing it is always enought to straightline concatenate the encoding and the string. However using the weak translation might result in behavioral difference between RO and  $\text{RO}_{\text{ts}}$ . In particular in the two following cases, in the  $\text{RO}_{\text{ts}}$  the outputs of this queries would be different with high probability but might differ more in the RO:

- The prefixing bit of the concatenated string prevents collision of random oracles queries, ie.  $\text{RO}(1\|s) \neq \text{RO}(0\|\hat{\sigma}(e))$  even if  $\sigma^{-1}(e) = s$  and simplifies the proofs. It would also be possible to do a straightline concatenation and bound the probability of collision.
- We also precise the lenght of the string to avoid different tuples collisions, ie.  $\text{RO}(0\|\hat{\sigma}(e)\|1\|s) = \text{RO}(0\|\hat{\sigma}(e)\|0\|\hat{\sigma}(e_2)\|1\|s_2)$  if  $s = 0\|\hat{\sigma}(e_2)\|1\|s_2$ .

We purposefully let the choice of length encoding open as it is not important for the rest of the proofs. One reasonable choice could be to use self-delimiting codes [31] such that one can even encode arbitrary length strings with only a small overhead. If algorithms are limited to polynomial time algorithms one can also use a fixed size encoding as the algorithm can only parse polynomial sized strings. The strong translation would be useful for [Theorem 10](#) to handle canonical translation of non polynomial time algorithm.

*Remark 2.* While using strong canonical translation makes the proof easier as it gives a perfect match between the type-safe algorithm  $\mathcal{A}$  and it's random representation translation, in the case of polynomial time algorithm it is enough to use the weak version. For  $Q \in \text{poly}(\lambda)$   $\text{RO}_{\text{ts}}$  queries made by  $\mathcal{A}$  in the worst case scenario  $\mathcal{A}$  we can partition the queries in two sets such that  $E := \{e_i\}_{i \in [Q]}$  and  $S := \{s_i\}_{i \in [Q]}$ . We can bound the probability of the two bad events described in [Remark 1](#) to  $C := \{\exists i, j \in [Q] : \sigma(e_i) = s_j\}$ . The probability that to find a collision between two set in the codomain of random injection  $\sigma$  is bounded by  $\frac{Q^2}{|S|} = \text{negl}(\lambda)$ .

**Theorem 9.** Given a random oracle  $\text{RO}_{\text{ts}}$ , for any PPT  $\mathcal{A} \in \text{TSA}^{\text{RO}_{\text{ts}}}[U]$  whose final output is of type  $\langle \text{bits} \rangle$ . The output distribution of  $\mathcal{A}^{\text{RO}_{\text{ts}}}$  and its weak canonical translation  $\mathcal{B} \in \text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$  is statistically close.

*Proof.* Observe that it is equivalent for type  $\langle \text{elem} \rangle$  variable to contain  $x$  or  $\text{label}(x)$ . An algorithm  $\mathcal{B}^{\text{RO}}$  simply replace variables with type  $\langle \text{elem} \rangle$  variable of type  $\langle \text{bits} \rangle$  corresponding to the encoding  $\text{label}(x)$ . Furthermore, as  $\mathcal{A}^{\text{RO}_{\text{ts}}}$  and  $\mathcal{B}^{\text{RO}}$  random oracles outputs are both uniformly random and as  $\mathcal{A}$  is a polynomial time algorithm, all the query to  $\text{RO}_{\text{ts}}$  preserve the same output distribution as queries to RO with all but negligible probability ([Remark 2](#)). It follows that the output distributions of  $\mathcal{A}^{\text{RO}_{\text{ts}}}$  and  $\mathcal{B}^{\text{RO}}$  are the statistically close.

**Theorem 10.** If  $Ch'$  is secure in  $\text{RR}^{\text{RO}}[U]_{\hat{\sigma}}$ , then  $Ch$  is secure in the  $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  model.

*Proof.* Let  $\mathcal{A}$  be an adversary against  $Ch$  in the  $\text{TSA}^{\text{RO}_{\text{ts}}}[U]$  model. Notice that  $(Ch', A')$  is the canonical translation of  $(Ch, A)$  and are complete (i.e., apart from oracle messages, there is a one-to-one correspondence between outgoing and incoming messages). Thus from [Theorem 9](#), the output distribution is identical. This contradicts hardness of  $Ch'$ .

**Theorem 11.** *If  $Ch$  is a single-stage game and  $\Pi$  satisfies  $Ch$  property in the  $\text{TSA}^{\text{ROts}}[U]$  model. Then,  $\Pi'$  satisfies  $Ch'$  property in  $\text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$ , where  $Ch'$  and  $\Pi'$  are canonical translations of  $Ch, \Pi$  with respect to a randomly chosen encoding  $\hat{\sigma}$ .*

*Proof.* We will proceed by contradiction. Let  $\text{RO}_{\text{ts}} : (\langle \text{elem} \rangle e, \langle \text{bits} \rangle s)^* \rightarrow \langle \text{bits} \rangle r$  and  $\text{RO} \langle \text{bits} \rangle s \rightarrow \langle \text{bits} \rangle r$ . Let  $\mathcal{B}^{\text{RO}}$  be a single stage adversary playing the game  $Ch' \in \text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$  and winning with probability  $\epsilon$ , which is non negligibly greater than  $t$ .

We assume that from the description of  $Ch$  and  $\Pi$  we can derive a deterministic predicate forward  $\text{fwd} : \langle \text{bits} \rangle s \rightarrow \langle \text{bit} \rangle b$  that is true if the bitstring should be forwarded to the random oracle and false if not. Such a function simply characterizes all possible lengths of random oracle queries that would be made by the challenger (given  $\lambda$ ). For example, for zkSNARKs via Fiat-Shamir transform, this would be the lengths corresponding to the partial transcript for all rounds in the protocol. Then in  $\text{Hyb}_4$  below, we use this fact to parse a query in bits meant for RO to a query in terms of  $\langle \text{elem} \rangle$  and  $\langle \text{bits} \rangle$  so that it can be passed to  $\text{RO}_{\text{ts}}$ .

We now construct an adversary  $\mathcal{A}^{\text{ROts}}$  for  $Ch \in \text{TSA}^{\text{ROts}}[U]$  which win with probability at least  $\epsilon - \text{negl}(\lambda)$ . Thus contradicting the fact that  $(Ch, t)$  is assumed to be hard.

Let's define  $S_{\hat{\sigma}} \subset \{0, 1\}^*$  the set of all possible label output by  $\hat{\sigma}$  and  $S_{\text{ROts}}$  the set of all possible output of  $\text{RO}_{\text{ts}}$ . We assume that it is possible to sample from the set  $S_{\hat{\sigma}}$  of encoding. We will use a key-value map table supporting many-to-many relations and assume that it is easy to test membership of an element in a table using the equality gate, we will use  $e \in T$  for an element  $e$  or  $s \in T$  for a string  $s$  as a syntactic sugar for such operation. Let's assume that the adversary can efficiently sample valid labels on its own with probability  $\frac{p}{|S|}$ , for example if the encoding is a random encoding of size close to cardinality of the group.

First, we make two observations that will be useful later:

- (1) Notice that  $\mathcal{A}$  can easily lazy sample new group elements by taking  $a \xleftarrow{\$} \mathbb{Z}_p$  and call  $e \leftarrow \text{label}(a)$  and check that  $e \notin T_{\hat{\sigma}}$ . Then  $\mathcal{A}$  can add  $e$  to  $T_{\hat{\sigma}}$ .
- (2) As  $\hat{\sigma}$  is a random injection the probability that it satisfies any bits relation to some specific labels is negligible (as such bit relations from uniformly chosen labels is independent of the group structure).

We will now proceed through a series of hybrid experiments to show that an algorithm  $\mathcal{A}$  as described in [Protocol 2](#) succeeds against  $Ch$  with probability  $\epsilon - \text{negl}(\lambda)$ . We start with  $\mathcal{B}$  interacting with  $Ch'$ , and in each hybrid change  $\mathcal{B}'$ 's behaviour so that after all the changes it is identical to how  $\mathcal{A}$  runs. At this point, we can switch from  $Ch'$  to  $Ch$ , with the canonical transformation happening inside  $\mathcal{A}$  instead of in  $Ch'$ .

**Hyb<sub>0</sub>:** This is the initial game where  $\mathcal{B}$  interacts with  $Ch'$  and wins with probability  $\epsilon$  by assumption.

**Hyb<sub>1</sub>:** This experiment is the same as **Hyb<sub>0</sub>** except that:

- If at any point  $\mathcal{B}$  sends a label  $\ell$  that was not explicitly given to him in some prior query or interaction with the challenger, sample  $x \xleftarrow{\$} \mathbb{Z}_p$  uniformly and set  $\text{label}^{-1}(\ell) \leftarrow x$  with probability  $\frac{p}{|S|}$  and set  $\text{label}^{-1}(\ell) \leftarrow \perp$  with probability  $1 - \frac{p}{|S|}$ .

As by observation (2) the probability that a bit encoding of  $x$  would be derived from previously queried  $\ell_i$  using bit operations is negligible and therefore  $\mathcal{A}$  can distinguish with only negligible probability. Therefore we have:  $\text{Adv}_{\mathcal{B}}^{\text{hyb}_1}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{hyb}_0}(\lambda) - \text{negl}(\lambda)$ .

**Hyb<sub>2</sub>:** This experiment is the same as **Hyb<sub>1</sub>** except that:

- The labeling function  $\text{label}(\langle \text{bits} \rangle x)$  or a new element in an OP query are lazy sampled.

We can follow observation (1) to realize the sampling. This is the same as  $\text{Hyb}_1$  from  $\mathcal{B}$ 's point of view thus:  $\text{Adv}_{\mathcal{B}}^{\text{hyb}_2}(\lambda) = \text{Adv}_{\mathcal{B}}^{\text{hyb}_1}(\lambda)$ .

$\text{Hyb}_3$ : This experiment is the same as  $\text{Hyb}_2$  except that:

- For every random oracle queries made by  $\mathcal{B}$  for a bitstring  $s$ :
  1.  $b \leftarrow \text{fwd}(s)$
  2. if  $b = 1$ : return  $\text{RO}(s)$
  3. else: lazy sample the random oracle answer.

This is the same as  $\text{Hyb}_2$  from  $\mathcal{B}$ 's point of view thus:  $\text{Adv}_{\mathcal{B}}^{\text{hyb}_3}(\lambda) = \text{Adv}_{\mathcal{B}}^{\text{hyb}_2}(\lambda)$ .

$\text{Hyb}_4$ : This experiment is the same as  $\text{Hyb}_3$  except that:

- instead of passing the query to  $\text{RO}$ , parse it as  $q = (\langle \text{elem} \rangle e_i, \langle \text{bits} \rangle s_i)_{i \in [n]}$  where  $n$  is the size expected by the challenger and  $\langle \text{elem} \rangle e_i$  are retrieved from  $T_{\hat{\sigma}}$ , for elements that are not in the table we apply observation (1).
- Forward  $q$  to  $\text{RO}_{\text{ts}}$

As both  $\text{RO}$  and  $\text{RO}_{\text{ts}}$  output distributions are uniform and forwarded queries always yield the same output this is the same as  $\text{Hyb}_3$  from  $\mathcal{B}$ 's point of view thus:  $\text{Adv}_{\mathcal{B}}^{\text{hyb}_4}(\lambda) = \text{Adv}_{\mathcal{B}}^{\text{hyb}_3}(\lambda)$ .

$\text{Hyb}_5$ : In this last experiment  $\mathcal{B}$  is a subroutine of  $\mathcal{A}$ . For  $\mathcal{B}$  this view is identical to  $\text{Hyb}_4$ . The canonical translation between  $\text{TSA}[U]$  and  $\text{RR}[U_{\hat{\sigma}}]$  happen inside  $\mathcal{A}$  instead of  $Ch'$ . Therefore,  $\mathcal{B}$  still win with advantage:  $\text{Adv}_{\mathcal{B}}^{\text{hyb}_4}(\lambda) \leq \epsilon - \text{negl}(\lambda)$ .

This is enough to conclude that  $\text{Adv}_{\mathcal{A}}^{\text{hyb}_4}(\lambda) \leq \epsilon - \text{negl}(\lambda)$  for any  $\mathcal{A} \in \text{RR}^{\text{RO}}[U_{\hat{\sigma}}]$ .

**Protocol 2:** Adversary  $\mathcal{A}^{\text{ROts}}$  against  $Ch$

**Initialization:**

- 1: Initialize an empty table  $T_{\hat{\sigma}}$  with key  $x \in \langle \text{elem} \rangle$  or  $\ell \in \langle \text{bits} \rangle$  and values  $\ell \in \langle \text{bits} \rangle$  or  $e \in \langle \text{elem} \rangle$
- 2: Initialize an empty table  $T_{\text{RO}}$  with key  $s \in \langle \text{bits} \rangle$  and values in  $r \in \langle \text{bits} \rangle$
- 3: Define  $D_{S,p}[T_{\hat{\sigma}}]$  as the distribution that outputs  $a \in \mathbb{Z}_p \wedge \text{label}(a) \notin T_{\hat{\sigma}}$  with probability  $\frac{p}{|S|}$  and  $\perp$  otherwise.

**Message from  $Ch$ :** ( $\{\langle \text{elem} \rangle e_i\}_{i \in [k]}, \langle \text{bits} \rangle s$ )

- 1: **for**  $i \in [k]$  **do**
- 2:   **if**  $e_i \in T_{\hat{\sigma}}$  **then**
- 3:      $\langle \text{bits} \rangle \ell_i \leftarrow T_{\hat{\sigma}}[e_i]$
- 4:   **else**
- 5:      $\langle \text{bits} \rangle \ell_i \xleftarrow{\$} S_{\hat{\sigma}} \setminus T_{\hat{\sigma}}$
- 6:      $T_{\hat{\sigma}}[e_i] \leftarrow \ell_i$
- 7:   **end if**
- 8: **end for**
- 9: Send ( $\langle \text{bits} \rangle \{\ell_i\}_{i \in [k]}, \langle \text{bits} \rangle s$ ) to  $\mathcal{B}^{\text{ROts}}$

**Message from  $\mathcal{B}$ :** ( $\langle \text{bits} \rangle \{\ell_i\}_{i \in [k]}, \langle \text{bits} \rangle s$ )

- 1: **for**  $i \in [k]$  **do**
- 2:   **if**  $\ell_i \in T_{\hat{\sigma}}$  **then**
- 3:      $\langle \text{elem} \rangle e_i \leftarrow T_{\hat{\sigma}}[\ell_i]$
- 4:   **else**
- 5:      $\langle \text{bits} \rangle a \leftarrow D_{S,p}[T_{\hat{\sigma}}]$
- 6:      $\langle \text{elem} \rangle e_i \leftarrow \text{label}(a)$
- 7:      $T_{\hat{\sigma}}[e_i] \leftarrow \ell_i$
- 8:   **end if**
- 9: **end for**
- 10: Send ( $\{\langle \text{elem} \rangle e_i\}_{i \in [k]}, \langle \text{bits} \rangle s$ ) to  $Ch$

**RO query:**  $\text{RO}(\langle \text{bits} \rangle s)$

- 1: **if**  $s \in T_{\text{RO}}$  **then**
- 2:    $\langle \text{bits} \rangle r \leftarrow T_{\text{RO}}[s]$
- 3: **else if**  $\text{fwd}(s) = 1$  **then**
- 4:   parse  $s$  as ( $\langle \text{bits} \rangle \ell_i, \langle \text{bits} \rangle s_i\}_{i \in [n]}$
- 5:   **for all**  $e_i$  **do**
- 6:     **if**  $e_i \in T_{\hat{\sigma}}$  **then**
- 7:        $\langle \text{bits} \rangle \ell_i \leftarrow T_{\hat{\sigma}}[e_i]$
- 8:     **else**
- 9:        $\langle \text{bits} \rangle a \leftarrow D_{S,p}[T_{\hat{\sigma}}]$
- 10:        $\langle \text{elem} \rangle e_i \leftarrow \text{label}(a)$
- 11:        $T_{\hat{\sigma}}[e_i] \leftarrow \ell_i$
- 12:     **end if**
- 13:   **end for**
- 14:    $r \leftarrow \text{RO}_{\text{ts}}((\langle \text{elem} \rangle e_i, \langle \text{bits} \rangle s_i)_{i \in [n]})$
- 15: **else**
- 16:    $\langle \text{bits} \rangle r \xleftarrow{\$} S_{\text{RO}}$
- 17:    $T_{\text{ROts}}[s] \leftarrow r$
- 18: **end if**
- 19: **return**  $r$

**Labelling query:**  $\text{label}(\langle \text{bits} \rangle x)$

- 1:  $\langle \text{elem} \rangle e \leftarrow \text{label}(x)$
- 2: **if**  $e \in T_{\hat{\sigma}}$  **then**
- 3:    $\langle \text{bits} \rangle \ell \leftarrow T[e]$
- 4: **else**
- 5:    $\langle \text{bits} \rangle \ell \xleftarrow{\$} S_{\hat{\sigma}} \setminus T_{\hat{\sigma}}$
- 6:   Add  $T_{\hat{\sigma}}[e] \leftarrow \ell$
- 7: **end if**
- 8: **return**  $\langle \text{bits} \rangle \ell$

**Group operation** **query:**  
 $f(\langle \text{bits} \rangle \ell_1, \langle \text{bits} \rangle \ell_2, \langle \text{bits} \rangle a, \langle \text{bits} \rangle b)$

- 1: **if**  $\ell_1 \in T_{\hat{\sigma}}$  **then**
- 2:    $\langle \text{elem} \rangle e_1 \leftarrow T_{\hat{\sigma}}[\ell_1]$
- 3: **else**
- 4:    $\langle \text{bits} \rangle a_1 \leftarrow D_{S,p}[T_{\hat{\sigma}}]$
- 5:    $\langle \text{elem} \rangle e_1 \leftarrow \text{label}(a_1)$
- 6:    $T_{\hat{\sigma}}[e_1] \leftarrow \ell_1$
- 7: **end if**
- 8: **if**  $\ell_2 \in T_{\hat{\sigma}}$  **then**
- 9:    $\langle \text{elem} \rangle e_2 \leftarrow T_{\hat{\sigma}}[\ell_2]$
- 10: **else**
- 11:    $\langle \text{bits} \rangle a_2 \leftarrow D_{S,p}[T_{\hat{\sigma}}]$
- 12:    $\langle \text{elem} \rangle e_2 \leftarrow \text{label}(a_2)$
- 13:    $T_{\hat{\sigma}}[e_2] \leftarrow \ell_2$
- 14: **end if**
- 15:  $\langle \text{elem} \rangle e_3 \leftarrow f(\langle \text{elem} \rangle e_1, \langle \text{elem} \rangle e_2, \langle \text{bits} \rangle a, \langle \text{bits} \rangle b)$
- 16: **if**  $e_3 \in T_{\hat{\sigma}}$  **then**
- 17:    $\langle \text{bits} \rangle \ell_3 \leftarrow T[e_3]$
- 18: **else**
- 19:    $\langle \text{bits} \rangle \ell_3 \xleftarrow{\$} S_{\hat{\sigma}} \setminus T_{\hat{\sigma}}$
- 20:    $T[e_3] \leftarrow \ell_3$
- 21: **end if**
- 22: **return**  $\ell_3$



## H Plonk and Marlin Typed Simulation Extractability

**The work of [33].** Obtaining zkSNARKs by compiling a polynomial interactive oracle proof (PIOP) via an extractable polynomial commitment scheme (PCS) is a common design paradigm [17, 27]. A very popular choice of PCS is the KZG commitment scheme [46] as it leads to constant size proofs. To optimize the compilation process towards getting smaller proof sizes, a common approach is to use the homomorphic properties of PCS for the so-called *linearization trick*. Essentially, this is in fact the version that several real-world implementations consider.

The work of [33] proved that KZG polynomial commitment scheme (with the linearization trick) is simulation-extractable in the algebraic group model and non-programmable random oracle model. Starting from this, they show that using KZG for the optimized PIOP compilation results in a simulation-extractable zkSNARK in the same model. As a corollary, they obtain that Plonk and Marlin are SIMEXT in the AGM + non-programmable ROM.

**Concluding UC security.** For us to conclude UC security for Plonk and Marlin given [33], all we need to do is type the result. For information theoretic reduction, notice that the algorithm consists of (1) parsing a list of bistrings and derive polynomials, (2) forward these polynomials, therefore it is trivially type safe. For computational reductions, we must do the following:

- Identify the underlying assumptions in the theorem statement. For computational assumptions, show that they are generically hard.
- Identify the sequence of hybrids for which indistinguishability reduces to one of the computational assumption identified before.
- For these hybrids, type the reduction, and check if it is model-preserving (it will usually be efficiency preserving).

For the specific case of [33], we will have to consider three main theorems – Theorem 4 (states NIZK and SIMEXT for the compiled zkSNARK), and Theorem 2 (states NIZK and SIMEXT for KZG with linearization trick, i.e., protocol  $\Pi_{lin}$ ), and Theorem 1 (states SIMEXT for KZG with multiple evaluations, i.e., protocol  $\Pi_{m-eval}$ ).

For Theorem 4, the computational assumptions are (1) Protocol  $\Pi_{lin}$  is NIZK and SIMEXT (which actually refers to Theorem 2) and (2) uniqueness of KZG proofs (hard under SDH). We can list the sequence of hybrids where these assumptions are used simply by observation. The hybrids using assumption (1) almost always build the reduction following the same outline: run the main adversary internally, forward queries to oracles of the underlying assumption if needed (eg. simulation queries), run the KZG extractor. This means that as long as the KZG extractor is type-safe (which it is as it performs no group operations), the reduction is model-preserving. Also notice that at some point during the reduction, the adversary has to program the random oracle while the simulator at the end doesn't. In this case programmability is a proof artefact and can be seen as something happening internally in the adversary.

For Theorem 2, the main computational assumption is OMSDH, Affine-MDH, and that protocol  $\Pi_{m-eval}$  is SIMEXT (which relates to Theorem 1). For the adaptive version of Theorem 1, the computation assumptions are: OMSDH, and affine-MDH. Finally, we remark that assumptions SDH and OMSDH are already shown to be generically hard in [33], while affine-MDH reduces to discrete-logarithm. It can be observed that the reductions to these problems are model-preserving since any group element computed in the reduction is done via group operations (and not by bit manipulation). To be explicit, we concretely type one of the main reduction, i.e., to OMSDH (Theorem 1, Lemma 3 in [33]) in Figure 6.

**Reduction to OMSDH.** The reduction maintains a local table  $T$  to record all queries to  $\mathcal{O}_f$  (Figure 3) made by the underlying adversary  $\mathcal{A}$  for performing group operations, and uses this to

obtain the coefficients for any group element present in the output of  $\mathcal{A}$ . All group operations are performed by querying  $\mathcal{O}_f$  and is not explicitly mentioned in the reduction below. On receiving the  $\text{crs}$  from the OMSDH challenger, it simply forwards it to  $\mathcal{A}$ , while at the same time recording these group elements in  $T$ . On receiving simulation query from  $\mathcal{A}$  on  $(c, x, y)$ , it computes  $\pi := (c - [y]_1)(s - x)^{-1}$ , where  $(s - x)^{-1}$  is computed by querying the  $\mathcal{O}_S$  oracle provided by challenger in the OMSDH game. On receiving a forgery  $(c^*, x^*, y^*, \pi^*)$ , given that all prior group operations were recorded in  $T$ , the reduction derives the group representation for  $c^*$  (represented as the polynomial  $F^*$ ), then computes  $Q, r$  such that  $F(X) := Q(X) + r(X - x^*)$ . Finally, it computes  $y^* := r^{-1}(\pi^* - [q(s)]_1)$  by querying  $\mathcal{O}_f$  and submits this to the OMSDH challenger.

**Figure 6:** OMSDH reduction:  $R^{\mathcal{O}}[\mathcal{A}](1^\lambda, \text{crs})$

**State:**

- 1:  $T$ : A table to store algebraic representation of each  $\langle \text{elem} \rangle$  variable.

**Setup:**

- 1: Parse  $\text{crs} = \langle \text{elem} \rangle(g_0, \dots, g_d)$ .
- 2: **for**  $i \in [0, d]$  **do**
- 3:     Store  $T[g_i] \leftarrow 1$
- 4: **end for**
- 5: Forward  $\text{crs}$  to  $\mathcal{A}$ .

**Simulation:**  $\langle \text{elem} \rangle c, \langle \text{bits} \rangle x, \langle \text{bits} \rangle y$

- 1:  $\langle \text{elem} \rangle[(s - x)^{-1}]_1 \leftarrow \text{Ch}.\mathcal{O}_S(1)$
- 2:  $\langle \text{elem} \rangle \pi := \langle \text{elem} \rangle(c - [y]_1) \cdot [(s - x)^{-1}]_1$
- 3: **return**  $\pi$

**Group query:**  $\langle \text{elem} \rangle h_1, \langle \text{elem} \rangle h_2, \langle \text{bits} \rangle a_1, \langle \text{bits} \rangle a_2$

- 1:  $\langle \text{elem} \rangle h \leftarrow \mathcal{O}_f(\langle \text{elem} \rangle h_1, \langle \text{elem} \rangle h_2, \langle \text{bits} \rangle a_1, \langle \text{bits} \rangle a_2)$ .
- 2:  $T[h] \leftarrow a_1 \cdot T[h_1] + a_2 \cdot T[h_2]$
- 3: **return**  $h$

**Forgery Extraction:**

- 1:  $(\langle \text{elem} \rangle c^*, \langle \text{bits} \rangle x^*, \langle \text{bits} \rangle y^*, \langle \text{elem} \rangle \pi^*) \xleftarrow{\$} \mathcal{A}$
- 2: Define  $F^*(X) := T[c^*]$
- 3: Define  $(Q(X), r) := F(X)/(X - x^*)$
- 4:  $\langle \text{elem} \rangle \pi_1 := \langle \text{elem} \rangle([q(s)]_1)$
- 5:  $\langle \text{elem} \rangle \pi_2 := \langle \text{elem} \rangle(\pi^* - \pi_1)$
- 6:  $\langle \text{elem} \rangle y^* := \mathcal{O}_f(\langle \text{elem} \rangle \pi_2, \langle \text{elem} \rangle g, \langle \text{bits} \rangle r^{-1}, 0)$
- 7: **return**  $(\langle \text{bits} \rangle x^*, \langle \text{elem} \rangle y^*)$